Department of Computer Science

Submitted in part fulfilment for the degree of BSc.

Providing a Pre-Partitioned and Partially Pipelined Parallel Process Prototype for Parsing XMI

Tobias Fox

2019

Supervisor: Dimitris Kolovos

Acknowledgements

I wish to acknowledge the help provided by my project supervisor, Dimitris Kolovos, particularly his outline of a successful tree partitioning algorithm that can be used to segment XMI. I would also like to thank Eclipse Modeling Project Lead Ed Merks not only for his work on EMF and its key literature, but also for his personal input on this project's implementation and his technical knowledge of EMF's capabilities. If in doubt, ask who wrote the book.

A final thought of gratitude lies with my family and friends who, noting my lack of availability, all tried their hardest to pull me away from this work.

Contents

List of Figures

List of Listings

List of Algorithms

Executive Summary

XML Metadata Interchange (XMI) is a standardised model exchange format supported by many Model Driven Engineering (MDE) tools, including the Eclipse Modeling Framework (EMF). The use of such tools often results in the production of large models that are serialised to XMI files of hundreds of megabytes, requiring several minutes to fully load on consumer hardware using EMF's native sequential loading process. One such example of this use is the reverse engineering of source code to UML which, for sufficiently large programs, can generate models with millions of objects and XMI with millions of elements. Over recent years processor clock speed has seemingly plateaued [1], being outpaced by the rise of multi-core and multi-processor outfits whose benefits, while fulfilling Moore's Law [2], are limited to parallelised software processes. Although previous research exists presenting several algorithms that use concurrency to load simple XML models into memory up to four times faster than their sequential equivalents [3], to the best of my knowledge there has been no attempt to do the same for the more complex XMI model loading process. A loading algorithm that not only takes advantage of such hardware improvements, but is scalable with them, is hence desired.

In this report I introduce the P4EMF algorithm and prototype implementation that allows the use of multi-core or multi-processor equipment to load serialised models at up to 2.2 times faster than the native loading process. Such speed gains are achieved through use of a fast sequential pre-parsing step that efficiently builds and partitions a document tree from an XMI file, before a combination of data and task parallelism is employed to concurrently load each partitioned XMI chunk. Throughout this process a simple two-stage pipeline is used to join each loaded partition into a master model which, after reconciling cross-partition references, is fully compatible with all sequentially-loaded models. This report fully describes each phase of this process not only in terms of an EMF implementation, but also in terms of high-level design choices relevant to general XMI processing.

The correctness of my prototype was tested by loading a number of XMI models with both EMF's sequential loading process and P4EMF. Their in-memory representations were then serialised back to XMI using EMF's default serialiser and compared for any discrepancies using Diffchecker [4]. Each model having exactly the same serialisation implies they have exactly the same in-memory representation.

Benchmarking P4EMF's performance is carried out by loading several models of varying sizes with both P4EMF and the sequential loading process on the University of York's *Viking* [5] high performance computing cluster, with access to 16 Intel Xeon Gold cores and 20GB of memory. The measured execution times of each of these processes show that while P4EMF performs worse with only a single loading thread, it can perform up to 2.2 times faster than the sequential loading process. Although still a substantial improvement, this is significantly less than the four times speedup Pan et al. encountered when investigating the parallelisation of XML parsers. As I suggest in Section 5.3.1, the best-case speedup of my prototype's architecture is little more than two times, a restriction due to the required sequential step of joining each loaded partition's model with the master, amplified by EMF's poor handling of each operation. My analysis also shows that P4EMF, and the loading process as a whole, is bottlenecked by the use of ID-based object referencing, and that models can be loaded significantly faster with relative path references. A comparison of ID-based and path-based referencing in XMI models can be seen in Listing C.4.

While the advantages of my prototype are clear, there are a small number of substantial limitations that reduce its applicability on consumer hardware; namely the requirement of at least five cores[1] and its four times greater peak memory usage when compared with the sequential loading process. Another disadvantage of P4EMF is the pre-parsing algorithm's extremely poor performance on XMI files of a certain structure.

An obvious extension to this report would be to find mitigations to the disadvantages listed above (and in detail in Section 6.1). Due to the noted performance improvements, I suggest that intensive testing of an improved prototype should be undertaken and smaller optimisations should be identified, with the overarching aim to convert this prototype into a product that can be bundled with EMF for use by the wider community. Further work is also suggested to reduce the impact that joining loaded partitions has on P4EMF's performance; this is the bottleneck of my prototype, caused by the way EMF handles moving the contents of one resource to another.

I see no ethical implications raised or ethical considerations that must be made due to any of the content of this report, and furthermore no content of my project or its findings will cause any harm to any individuals or organisations. Eclipse and EMF were used under the EPL open-source license agreement, and the large XMI data sets used throughout my analysis were obtained with permission from my supervisor.

[1] A minimum of four threads are required if ID-referencing is used; for speed improvements over the sequential process at least five threads are needed. Use of path-based referencing requires a minimum of two threads, with more providing performance benefits. In all cases there should be at least the same number of available processor cores as there are threads.

1 Introduction

Extensible Markup Language (XML) was introduced as a simplified profile of ISO Standardised SGML in 1998 [6] to accommodate an anticipated rise in online data communications. While XML usage fulfilled its intended purpose serving online data interchange, it has also become the backbone of many file formats such as those used by Microsoft Office [7] and SQL Server [8]. XML Metadata Interchange (XMI) allows simple XML-based expression of Model Driven Engineering (MDE) models and metamodels and was standardised in 2003 by the Object Management Group [9], and in 2005 by ISO. Currently one of its major uses is in the serialisation of UML data [10], and due to its platform and software independence [11] all major model management software languages and tools provide compatibility with XMI to at least an import/export level, with those based on the Eclipse Modelling Framework (EMF) providing much greater access [12].

Established XML parsing algorithms are inherently sequential therefore, perhaps unsurprisingly, there has been plentiful research into multi-threaded implementations [13], [3], [14], [15], [16]. To the best of my knowledge, however, there are no publicly available reports documenting the parallelisation of XMI parsers, something I find concerning given the continuously increasing volume of exchanged data and the noted unscalability parsing it sequentially. While the processor clock speed gains of recent years have arguably plateaued [1], Moore's Law continues to be fulfilled by the rise in multicore and multi-processor systems [2] whose availability on the consumer market leads towards a multi-threaded solution that is not only scalable with data but also with hardware improvements.

The next section begins by introducing the technicalities of the XMI format and its in-memory representation in EMF, the main forms of parallelisation, and previous relevant research. Section 3 follows by describing the structure and abstract design of the P4EMF[1] algorithm with relevant diagrams, before I discuss its concrete implementation in Section 4. In Section 5 I aim to convince the reader of the success of P4EMF by benchmarking the native loading process against my multi-threaded implementation with real-life XMI samples of varying sizes, before finalising my report with conclusions and recommendations of further work.

[1] Pre-Partitioned Pipelined Parser for EMF.

2 Background

2.1 Metamodels and models

Model Driven Engineering (MDE) is an abstracted view of traditional system development processes, removing algorithmic concepts in favour of *activities* that describe part of a particular domain. Other than a standard format and structure, its main benefit is total ambivalence towards particular applications or platforms resulting in maximised compatibility between descriptive models and their usage on different systems. An obvious use case of this is representing systems using UML [17].

Such a model is represented in two distinct parts with one-to-many relationship: the first is in abstract syntax as a Metamodeling names of attributes and their associated types, with the counterpart being the concrete set of information in the format of the referenced Metamodel. While each part can be communicated independently (for example, as an XMI text file), every Model has a dependence on its Metamodel being registered by the client software before it can be loaded itself.

2.2 XML and XMI

XML is a mark-up language designed to encourage data communications in a simple, human readable format [6]. Unlike other tag-based languages (eg. HTML), XML consists of only a small number of simple rules that lead to a well-formed (i.e. valid) subset of the language. In XML, any element name can be defined on the fly, as can its attributes and values provided values are within quotes, every opening tag has a matching closing tag, and element names do not begin with a number or 'XML' [18], although restrictions can exist in the form of an XSD schema.

```
<?xml version="1.0"?>
<university>
    <subject name="Computer Science"></subject>
    <subject name="Fine Art"></subject>
</university>
```

Listing 2.1: Sample XML document

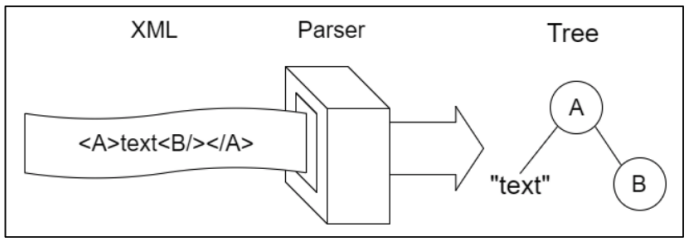

Figure 2.1: Visualisation of a DOM object tree parser, from [21].

As purely an extension of XML, XMI is fully compatible with the XML standard and thus is handled in much the same way. Unlike XML, however, XMI can handle many of the features associated with Object-Orientated Programming, including instance relationships, hierarchy and inheritance [19, p. 7]. While an XML parser is typically at the heart of XMI parsers [19, p. 9], without any alteration *special* reserved attributes like `xmi:id` would be treated as standard XML attributes, with references to other objects semantically disappearing.

```
<?xml version="1.0"?>
<xmi:XMI xmi:version="2.0" ... >
    <university subjects="s1 s2"></university>
    <subject xmi:id="s1" name="Computer Science"></subject>
    <subject xmi:id="s2" name="Fine Art"></subject>
</xmi:XMI>
```

Listing 2.2: Sample XMI document

2.2.1 DOM Parsing

Document Object Model parsing is an XML loading technique that consists of building a tree of the entire document where each node contains the loaded equivalent of each element. While this process has little overhead, a major drawback is that the entire document is initially loaded into memory at a cost of up to ten times the size of the original file [20]. Given documents with millions of elements and where file size regularly approaches hundreds of megabytes, it would be impossible to load such models on consumer hardware. DOM parsing is therefore unscalable with memory and indeed the use of a SAX Parser is recommended for XML documents with a size greater than *several megabytes* [16].

2.2.2 SAX Parsing

The other major parsing method is SAX (Simple Api for Xml). Here the XML document is processed element by element by a state-machine which

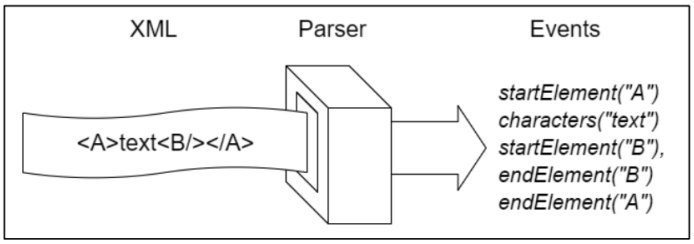

Figure 2.2: Visualisation of a SAX streaming parser, from [21].

raises events depending on the conditions of each transition. On receiving each event an API style event-handler executes the relevant functions to construct an in-memory representation of the model [22]. Contrary to DOM, SAX allows the parsing of streamed files and those of arbitrary length. For small XML files these callback overheads make this approach less efficient than DOM parsing, but for larger documents, especially when XMI is considered, a deliberate decision to use SAX parsing is often made as can seen by its usage in most open source XMI parsers including implementations used by SDMatrix [23], ArgoUML [24] and EMF itself [25].

2.3 Parallelism

Parallelisation is the process of transforming classical sequential programs into concurrent applications that take advantage of modern multicore and multiprocessor hardware by running several processes simultaneously [26]. An obvious requirement for parallelisation to have any useful benefit is the availability of multiple processors or cores; simply spawning multiple threads of execution on a single processor will likely increase execution time due to the overheads involved. Similarly, the memory requirements of a multi-threaded program must be considered, as they may be considerably higher then that of its sequential execution.

2.3.1 Task Parallelism

Task parallelism describes parallelism encountered at a logical or algorithmic level where a number of tasks are ran simultaneously and may join at some point in their executions, as exemplified in Figure B.1. It requires that the number of tasks to be run is fixed and the output of one task never reaches the input of another [27]. While task parallelism is a simple way of increasing system efficiency, according to Gordon, “it is not sufficient as the only source of parallelism” [27] as the amount of parallelisation is proportional only to the number of independent tasks, which is typically rather small.

2.3.2 Pipeline Parallelism

For systems containing processing tasks that depend on the output of previous tasks, parallelisation can be achieved through the use of pipelining where such tasks (AKA stages) are overlapped in threads that continuously execute a single task each. While for individual inputs pipeline parallelism provides no benefit (and may well increase processing time), with N stages and a constant flow of data, the system throughput can become close to N times higher than the sequential equivalent. Implementations of pipelining require a unidirectional flow of data between every stage and its successor, which must share no common state.

2.3.3 Data Parallelism

Data parallelism is a form of parallel execution where the amount of concurrency is proportional to the size of the input. For example, given a system which independently processes n inputs, maximum data parallelism could be offered if each input was processed on a different core or processor. Conditions for data parallelism are stringent, requiring that each task must not alter the state of its execution [27] or the context in which another task could execute. Furthermore, with data paralellism there is a real risk of communication and synchronisation latencies dominating the system if its utilisation is too high. A way of controlling this is to use an upper limit on the number of threads that can be spawned using a thread pool (for example, Java's `ExecutorService` [28]).

2.4 Existing XMI Loading Process in EMF

2.4.1 Data Structure

EMF is a Java framework based on Eclipse to aid with many MDE operations including code generation. Just as every Java object inherits from `Object`, every EMF metamodel is derived from `Ecore` - the model used to represent models [29, p. 17], which acts as every model's (and metamodel's) metamodel. `Ecore` consists of several basic properties that each metamodel must possess including `EClass` which represents the class of the element including an `EAttribute` list of attribute values and their types (`EDataType`) and an `EReference` list which refers to other `EClass` objects. A sample metamodel which can be seen conforming to this structure is present in Listing C.1.

A model must be loaded into a `Resource` to persist in memory, and is rep-

resented as an EObject instance consisting of its contents (eContents), links to its metamodel (eClass()), container object (eContainer()), and resource (eResource()). A loaded (meta)model is encapsulated by an instance of EPackage, the root element of its serialisation [29, p. 118], with all other content considered as this object's containments. *Containment* is an important concept in EMF meaning describing when EObjects are contained within other EObjects [29, pp. 111-112], accessed through the tree of *containment references* EObject.eContents(). From a serialised XMI view, the contents of a loaded Resource maps to the root of the XMI schema and the children of the root node exist in the root EObject's eContents as EObjects themselves.

2.4.2 Parsing Algorithm

Given an XMIResource, the deserialisation of its referenced XMI file is performed by Resource.load([options]) using EMF's native SAX parser with specified (or default) SAX handler. The following algorithm is as detailed in [29, pp. 469–470].

For every event the parser produces (eg. StartElement, EndElement) a callback is made to the relevant method in the SAXXMIHandler handler, where more of the model is constructed. The complex logic of loading XMI resides in this handler, which keeps track of its current position in the document as well as maintaining stacks of parsed elements, attributes and objects of the model in its global state. When the start of a new element is registered, the way the resource is updated is dependant on the handler's current state and the *namespace URI* of the handler's current element - these determine which Ecore package to locate and thus how to handle anything from its attributes to child elements. When attributes are parsed, a complimenting Ecore feature is obtained; in the case this feature is an EAttribute then the value is set, otherwise if the feature is an EReference then the ID of the corresponding object is located and set, or if the referent object has not yet been parsed (i.e. is a *forward* reference) the reference is flagged and set aside. This process iterates until the parser scans the end of the document, where the handler makes a final pass through the resource to tie up any forward references.

There has been little research into the time and space complexity of either XML or XMI parsing. One study suggests, through measurement of processing varying XML documents, that for small files of less than 100KB the average SAX parsing cost is approximately 175,000 instructions per KB of data [30]. Nicola and John also found that as XML documents increase in size, in the context of loading a document, the proportion of processor time spent parsing increases with respect to validation and reference checking, to the point where for an XML file of 1MB nearly half of loading time is

plain-text parsing, as opposed to other kinds of processing. While I would assume this is not the case for XMI parsing (which has a much higher focus on object relationships and reference reconciliation) this intensifies the requirement for such a parallel parser to exist.

2.5 Parallelisation of SAX-based XML Parsers

Existing research to parallelise XML parsers focusses mainly on data parallelism, as well as novel and more experimental techniques. One approach [13] that halved parsing time on four threads across two processors introduced a fast sequential 'pre-parsing' step which by use of Pushdown Automata quickly determined subgraphs within the XMLs simplified tree structure. This was performed using either *static* or *dynamic* partitioning of the tree to produce balanced XML chunks by depth-first search with either one sequential thread or many concurrent threads respectively. Such chunks were then loaded in parallel before being joined by a sequential process. When compared with sequential parsing, Lu et al. found this approach performed up to 3.5 times faster, although when including the considerable overhead of the pre-parsing phase its speedup was reduced to 2.5 with four threads.

Pan et al. [3] proposed a hybrid solution that combined data parallelism with pipelining, resulting in over four times speedup (with eight threads). The first step of this process "Speculatively Parses" arbitrarily sized chunks of the XML file to form a set of potential models (capturing the ambiguity of each chunk's position in the model) with data parallelism. After potential models are determined for all chunks, a sequential "Chunk Resolution" step identifies the single meaning of each chunk, passing each result to a pipelined task that forms a buffer of the structures useful for SAX callbacks. Once all chunks are fully processed, a final step resolves any outstanding inter-chunk references before making all the required SAX callbacks in their correct order. Without the use of any pre-processing, this method avoids the overhead associated with the previous method alluding to its higher performance gains.

A final, more novel, proposal [31] to improve XML parsing performance uses an automatic paralellisation tool to do the all of the heavy lifting. While the authors of this implementation saw speed improvements of up to 3 times, they accept that its limitations may be too great for this approach to be useful in a general XML parser setting.

3 Design

My parallel approach to parsing XMI comes in the form of P4EMF, a model loading algorithm that employs pipelining as two-stage interface between data-parallel and sequential tasks. To take advantage of data-parallelism there first needs to exist a set of inputs that can be processed individually. The generation of such chunks requires at least a preliminary pass through the document, and as each chunk must be well-formed (i.e. conforming to the XML syntax rules) it is not sufficient to build these partitions on the fly. Therefore, with inspiration taken from Lu et al. [13], I introduce a fast sequential *pre-parsing* step to form a basic representation of the XMI file that allows the construction of well-formed XMI chunks. Once loaded, chunks must also be joined in a sequential *post-parsing* step to form a master resource; a step which can be pipelined with the previous loading step. This process may be better understood with the help of Figure B.4, and is simply outlined as follows:

1. Form a simple document tree of the XMI file.
2. Partition the tree and build valid XMI chunks of less than x elements.
3. Load chunks concurrently using the existing EMF loading process.
4. Join loaded models to form the complete model.

3.1 Pre-Parsing

The goal of introducing a pre-parsing step is to form a basic structure that can be used to construct smaller XMI chunks to be loaded concurrently. It should be noted that pre-processing is an overhead cost; its performance impact should ideally be negligible.

My pre-parsing algorithm sequentially executes over three phases - the synchronisation costs of either reading a text-file or constructing a tree data structure with multiple threads are both prohibitively high, so a process that does both of these is bound to perform better sequentially. As a result this stage was designed to require only a single full pass through the document, and consequently processes as little data as possible. According to Amdahl's Law [32], which I will use to analyse my solution in Section

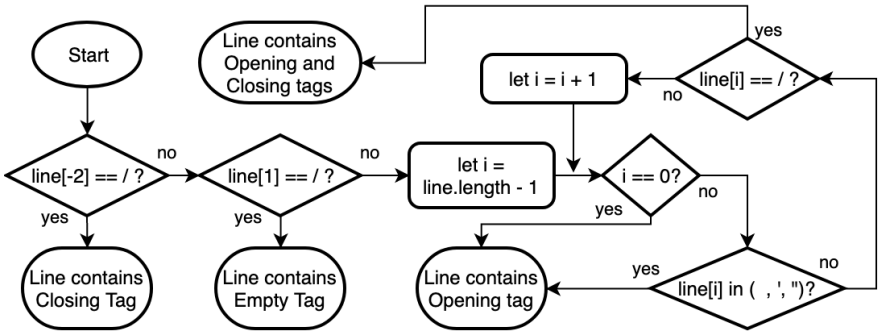

Figure 3.1: A flowchart of the algorithm that can classify a line of XMI provided my assumptions stand.

5.3.1, greater speedup is possible with a greater proportion of execution time that can benefit from data-parallelisation. This was relevant when designing this algorithm as a concious effort was made to delegate as much work to the non-sequential phases of P4EMF.

Phase One: Line Identification and Tree Construction

As XML is not a regular language “it cannot be parsed by a finite-state automaton, but rather requires at least a push-down automaton” says Lu et al. [13]. Indeed in order to identify even the most basic tree structure of an XML document a push-down automaton is required. The first phase of my pre-parsing algorithm tackles this issue by efficiently identifying the characteristics of each line of XMI, before using this knowledge to construct a tree-like data structure of nodes.

Lu et al. describe a state machine that iterates over each character in the file, transitioning into states that identify comments, tags, attributes and other structural features of XML. I was able to drastically simplify this approach by making several assumptions about the incoming data with respect to the Specification Rules of XML [33]. Firstly, I assume that the first line of the file will always contain the `<?xml...?>` declaration of XML (which is ignored). Secondly, I assume that every following line will consist of either a. exactly one opening tag, b. exactly one closing tag, c. exactly one opening tag and its corresponding closing tag (with optional content within) or, d. exactly one empty tag. The final assumption I make is that there are no comments, CDATA or Processing Instructions within the document, and while leading whitespace is allowed, there can be no trailing characters. Although these assumptions seem highly restrictive, each is valid for all models generated by EMF’s default serialiser.

This set of simple assumptions allows the reduction of the state machine formed by Lu et al. to the state machine described in Figure B.5. Furthermore, assuming a data representation where the complexity of reading

characters from the end of the line is roughly equivalent to reading from the start, many fewer comparisons are required if the algorithm shown in Figure 3.1 is used. While it should be clear that based on my assumptions this process is sufficient to identify each line, one concern may be that it is not capable of identifying and reporting invalid XMI documents (for example, a document containing nothing but closing tags). While this is true, as each partition is to be loaded using the existing parsing algorithm, the validity of each partition will confirmed at a later stage.

A weighted tree can be formed using a fraction of the resources required for constructing the model's DOM tree by executing the following operations based on each line's type (a, b, c, or d) after the root node is set:

- a: add a child to the current node and walk to the new child.
- b: walk to the current node's parent.
- c or d: add a child to the current node.

Phase Two: Static Partitioning

Once the minimum document tree is formed the next task is to decide where it should be *cut*. The optimal solution is for there to be the same number of partitions as there are available threads and for each partition to require the same amount of time to load. This ensures that all available threads are always busy doing meaningful processing and, theoretically, suggests a potential speedup factor of m, where m is the number of available threads (assuming the pre-parsing and post-parsing steps have negligible cost).

Dividing a tree requires that each node has a weight and a maximum-weight per partition is set. An optimal, albeit unrealistic, weighting for this tree could be described as the 'time required to load', but for a variety of reasons a heuristic-based approach is more efficient. Two values best convey the essence of the optimal weight; the number of children a particular node possesses (including grandchildren, great grandchildren etc.), and the length of the node element's line. Although the former does not attempt to approximate the time each node requires to load (unlike the latter), it is optimal assuming all elements take the same time to load. Given the repetitive format of XMI and the speed at which each individual element is parsed, this assumption is not too far from the truth and is therefore what I will use in my algorithm. Consequently the maximum number of elements in each partition, x, is determined by dividing the total number of elements in the tree[1] by the number of threads available for parsing, m.

While an efficient algorithm to equally partition weighted trees exists [34],

[1] This can also be seen as the number of children held by the root node plus one.

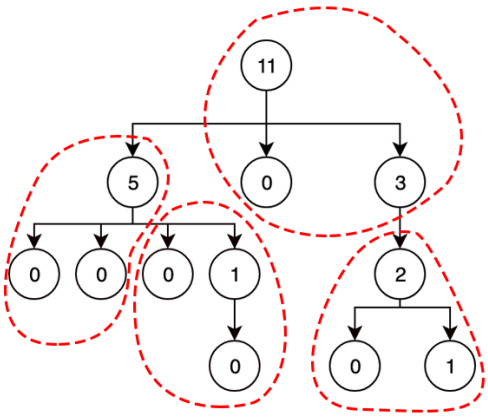

Figure 3.2: Diagram showing a tree optimally partitioned for processing with $m = 4$; $x = 3$, displaying the weight of each node.

it is unsuitable for our purposes. Lukes' algorithm generates partitions that can contain parental nodes along with any number of their children (for example, as seen in Figure 3.2, and while this helps its partitions to be more equally sized, the complexity of the process that merges each partition into a master resource is increased. Instead, my algorithm, as expressed in Figure 3.3, results in partitions containing either single nodes that have more children than can fit in a partition, or sets of sibling nodes and all their children as seen in Figure 3.4.

My partitioning algorithm performs well, with a time complexity of $\mathcal{O}(n)$. At its worst (the tree show in Figure B.6) the algorithm requires $2n - x$ operations as every parent node is too big to fit in a partition, while its best-case performance (the tree in Figure B.7) requires only $m + 1$ operations to partition as each of the m children of the root node can be added to partitions without needing to traverse their children.

Phase Three: Satisfying Well-Formedness and XMI Constraints

In order to load each partition as a separate entity each XMI chunk must conform to the requirements set out by the parser. For EMF's native parser, and I suspect every ordinary XMI parser, the minimum requirements are that each chunk is well-formed with respect to XML's syntactical rules and compliment to a metamodel. As it stands, the chunks of XMI that can be extracted from our newly-formed partitions have no guarantee of having either of these qualities - partitions containing single nodes produce well-formed chunks that may not be compliant to a model if they represent anything but the root node; partitions containing sets of siblings cannot produce well-formed chunks (XML specification rules dictate the existence of a single root node [33]) while model compliance shares the same fate.

To combat this, another phase is introduced to construct strings to insert

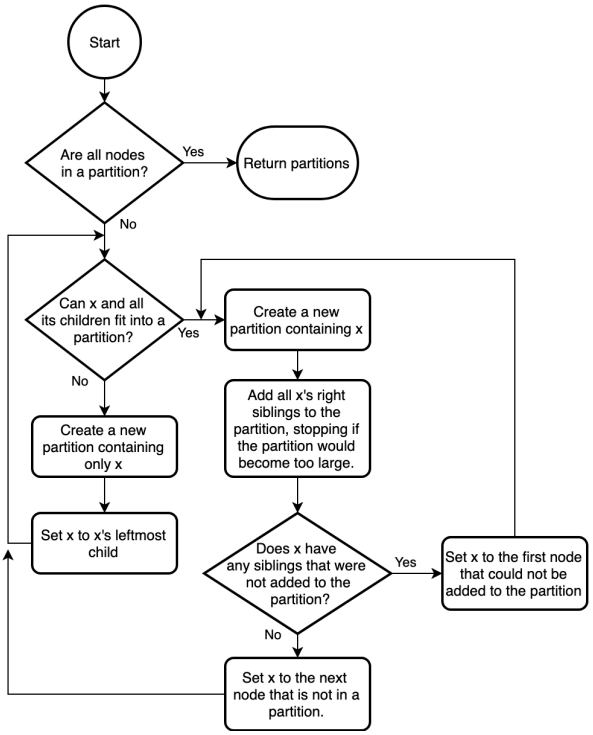

Figure 3.3: Flowchart depicting my tree partitioning algorithm, where x is initially set to the root node.

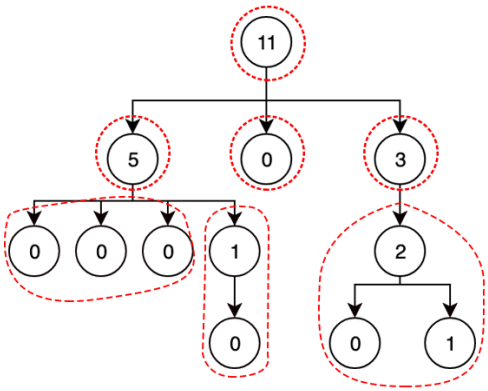

Figure 3.4: Diagram showing a tree partitioned for processing with $m = 4$; $x = 3$ by P4EMF's partitioning algorithm, displaying the weight of each node.

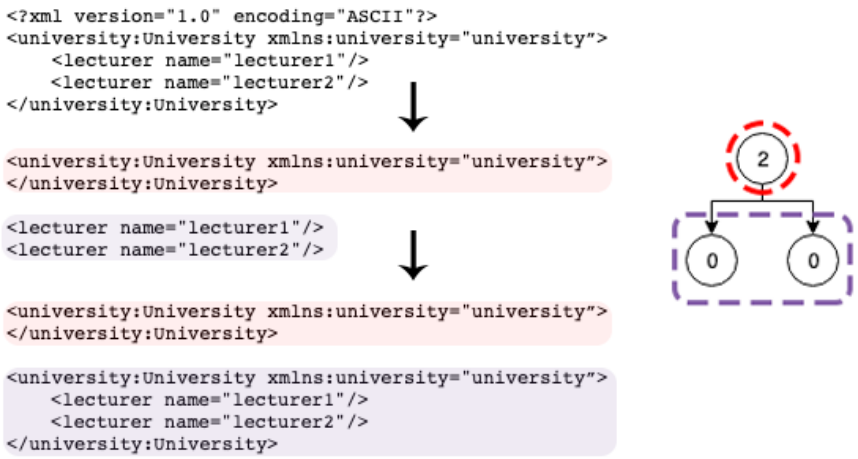

Figure 3.5: Example of the output of each stage of my pre-parsing process, showing a source XMI document, the document tree build in phase one, the two potentially invalid XMI chunks of the partitions formed in phase two, and the *fixed* partitions from phase three.

to the front and rear of chunks to ensure they are well-formed and model-compliant. Such *header* and *footer* strings contain nothing except essential data, consisting of the concatenations of the opening and closing tags (respectively) of every parent of the first or only node in the partition up to the root of the tree. Although required properties are a possible feature of XMI elements, importantly there is no such thing as required children according to the XMI Specification [35]. Given a partition that divides siblings of a node, the lack of such property implies that provided each chunk has the same header & footer elements, XMI validation cannot fail for this reason. Moreover, this phase guarantees that all partitions will conform to their models if they did previously, as well as clearly satisfying the requirement for a single root node.

This phase is extremely fast in typical uses where a small number of partitions are formed. Its worst-case execution grows at $\mathcal{O}(n^2)$, however, requiring close to $\frac{(n-\frac{n}{m})(n-\frac{n}{m}+1)}{2}$ operations for the sample structure in Figure B.6, caused by the large number $(n - \frac{n}{m})$ of partitions and the distance required to traverse to the root node for each partition.

3.2 Resource Loading

This step is the core of the loading process and the one that brings about the performance improvements of data parallelisation. It is this step's responsibility to construct the complete stream of well-formed XMI that represents each partition which are then loaded using an efficient parsing algorithm over a fixed but configurable number of threads. Once loaded,

resultant models are stored as a list of sub-resources in the same order as their partitions. As an optimisation, a two stage pipeline between this step and the post-parsing step can be formed, as both are execute with independent states and posses a unidirectional data flow.

The loading of each partition is performed over two phases: the first builds the partition's well-formed XMI stream, the second loads it. For this reason, and as my partitioning algorithm produces more partitions than there are available threads, there is an argument to be made about the optimal configuration of such tasks. Forming a single task from the composition of both phases is the simplest and most intuitive solution, but it is by no means optimal. Another approach is to run each phase as separate tasks (albeit with dependencies) according to a scheduling scheme such as Round Robin. While the optimal interleaving of tasks cannot be found as there is no way of knowing how long each chunk will take to load, it can certainly be approximated heuristically using, for example, the size of each chunk.

Synchronisation is an important aspect to consider when exploiting concurrency and is required to ensure reliable results when many threads share data. Assuming the existing loading process is 'thread-safe' [36] and both loading phases combine into a single concurrent task, each partition can be loaded with minimal synchronisation as the tasks are completely independently of one another. Delegating loading tasks to a number of threads, however, requires either the main thread to wait for any other thread to complete its current execution if there are still chunks to load, or the use of a thread pool and a queue of partitions shared between each member. Once a model is loaded, adding it to a shared variable *must* be performed under mutual exclusion to avoid any nondeterministic behaviour related to the shared variable's simultaneous access. Traditionally this has been implemented using semaphores and monitors [37], although most high-level programming languages provide their own constructs. If pipelining is introduced then synchronisation becomes more complex; the results of each successive stage must also be stored in some shared variable as previously described. Finally, once all partitions are loaded, some form of synchronisation needs to be used to terminate each spawned thread and resume the main program's path of execution.

3.3 Post-Parsing

The output of P4EMF is a loaded model identical to the one produced by the native loading process. This step consolidates the loaded chunks by joining their models in accordance to each partition's position in the document tree, before finalising cross-partition references. While most of the details of this step will depend heavily on its implementation, it can be divided into

two phases with the former executing as the second stage of the loading pipeline as described previously and the latter running sequentially after the pipeline completes. All post-parsing processing can be considered an overhead so, just like the pre-parsing step, this should be minimised.

Phase One: Sub-Tree Reconciliation

This phase begins as soon as the first resource is loaded and aims to construct the original document model partition by partition. For each loaded sub-resource in its original partition order, this phase splices its contents into the correct position in the master resource tree. While it is impossible to achieve data-parallelism here, task-parallelism may be possible for any other implementation-specific operations that are applied on each iteration, provided that each task is fully independent.

Joining resource trees in this context is a relatively simple and efficient process, thanks to my partitioning strategy which ensures that there are no out-of-order nodes; i.e. given an ordered list of partitions and a sub-tree t of any partition p, it is guaranteed that the parent node of t is in a partition before p, and that in all partitions before p, none of t's right-siblings are present. This means that once the parent node of any partition's contents is located in both the master its sub-resource, the partition's contents can be spliced into the master by simply appending the sub-resource parent node's children to the right of the master's parent node's children.

Phase Two: Cross-Partition Reference Handling

The final phase of P4EMF is a fast sequential phase to handle any cross-partition references. Once the sub-trees have been merged into a 'master' model, such references can be seen as analogous to forward-references, and indeed that is exactly how they will be treated. Existing XMI parsers generally handle these at the very end of the loading process to resolve and differentiate between forward references and references to non-existent objects. Despite solutions to this being implementation specific, the general approach is to form the context required by the existing forward-reference handling routine, and execute it against the reconstructed model. This final phase guarantees that legitimate cross-partition references are resolved correctly, and that invalid references are always identified.

4 Implementation

My implementation of P4EMF follows the design set out in the previous section, organised into a small number of classes as is visualised in Figures B.8 and B.9, which compare the structure of both EMF's native loading process and P4EMF. To summarise in words: when a `ParallelXMIResourceImpl` instance is created the referenced XMI file is pre-parsed by an instance of `XMIPreParser`, forming an `XMIPartition` list on the resource. Calling `ParallelXMIResourceImpl.load(..)` instantiates `SubXMIResourceImpl` objects for each `XMIPartition` then constructs and parses their XMI chunk's stream over a number of threads. Once partitions are loaded they are joined together as the `contents` of the `ParallelXMIResourceImpl` object before cross-partition references are handled in the master resource. The parallelisation and threading of this implementation is visualised in Figure B.10.

4.1 Pre-Parsing

Phase One: Line Identification and Tree Construction

As the content of the XMI document will be accessed a number of times, the first step of my implementation is load it into memory as an `ArrayList` of strings[1]. The bulk of this phase comes in the form of the `XMIPreParser.indentifyLine(..)` which identifies the type of each line (using the process described in Section 3.1) and constructs the appropriate tree of `XMINode` objects initially storing nothing but their line number, type and a reference to their parent node. As opening tags are detected children are added as described previously; as closing tags are detected each node's weight (`recursiveChildCount`) is calculated based on the sum of this value for all its children. This process is extremely quick as although it does consider a large proportion of characters in the document, for each line there is a maximum of one insertion and one step over the tree and all data insertions are performed at $\mathcal{O}(1)$. Figure 4.1 shows the in-memory representation of the tree formed from some sample XMI data.

[1] While this immediately introduces a considerable overhead it is much faster than reading data from the disk over and over.

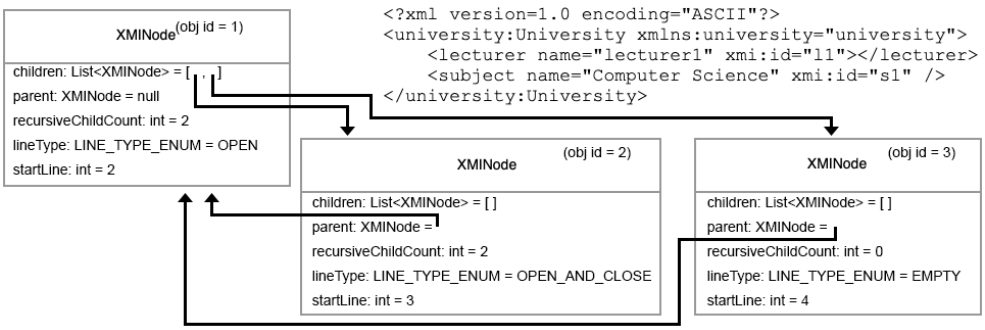

Figure 4.1: A simple example of the `XMINode` tree formed by the first phase of pre-parsing for the sample of XMI.

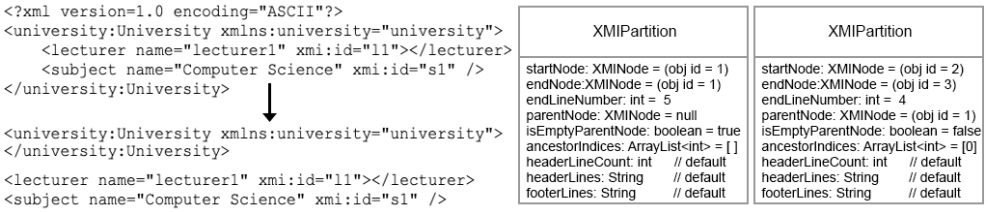

Figure 4.2: A simple example of the two `XMIPartition`s formed by the second phase of pre-parsing for the same sample of XMI and XMINode objects described in Figure 4.1.

Phase Two: Static Partitioning

The static partitioning of the tree-structure built in the previous phase is performed exactly as described by Algorithm 1 in accordance to the design with very little alteration. The output of this phase (produced by `XMIPreParser.getPartitions()`) is an ordered list of `XMIPartition` objects describing the first and final nodes of the partition, the stack of index positions required to traverse from the root node to the start of the partition, and a helper property that says whether or not the partition contains only a single node as shown in Figure 4.2.

Phase Three: Satisfying Well-Formedness and XMI Constraints

The final phase of pre-parsing constructs the strings that will pad the partitioned chunk in order to construct valid XMI, according to Algorithm 2 and the design set out. As this process uses the cached version of the original document there is very little time spent on I/O and although string manipulation is an expensive process, when multiplied by the limited number of partitions the cost of this operation can be near negligible[2]. My assumptions make this process even more efficient, as finding and inserting the opening tag of each parent node can be reduced to inserting the entire

[2]This is not always the case; in Section 5.1 I illustrate significant issues with this phase.

line belonging to it without any further processing. Furthermore, as XML closing tags strictly contain the element name surrounded by “</" and “>" they can be constructed from this line very simply.

4.2 Resource Loading

Loading the document into memory is achieved, as ever, by calling `Resource.load([options])`. In `ParallelXMIResourceImpl` this method is overridden to form and load each partition as a `SubXMIResourceImpl` in a configurable number of threads. Java's `Executors.newFixedThreadPool(`m`)` and `ExecutorService` are used to handle this parallel execution by creating and maintaining a thread pool and shared queue of `Runnable`s that limits the number of active threads to m.

As described in my design there are two phases to loading each sub-resource. Firstly, each partition stream must be constructed by several string and array manipulations as seen in the Java snippet in Listing A.1. What would be a very expensive process if using the original data-stream or `Arrays.copyOfRange(..)`, is made rather more efficient by using the cached `ArrayList` storing the original document and its `subList(..)` method which allows chunks to be formed without copying any data. On a similar note of optimisation, inserting this sub-list into a list already containing the header string avoids an $\mathcal{O}(n)$ insertion to the front, leading to negligible cost given the small number of partitions. The bottleneck of this step is generation of the `InputStream` which unfortunately requires each line of the chunk to be processed individually.

Before each `InputStream` can be loaded, new `SubXMIResourceSet` and `SubXMIResource` objects are created. Each new resource is then loaded with reference to the `InputStream` using the native loading process, albeit with the `DefaultHandler` overridden to return an extension of `SAXXMIHandler` that skips any calls to `handleForwardReferences(..)`[3]. While my design noted that efficient task scheduling is important to minimise execution time, in practice it makes very little difference as the inefficiencies encountered are far less than other overheads that limit the P4EMF's performance. For this reason my implementation simply executes both phases sequentially in the same `Runnable`, which are scheduled by `ExecutorService.Execute(..)`.

As an optimisation, my resource loading `Runnable`s also execute two more operations with task-parallelism. While these do not affect the performance of each load it allows their routines to run concurrently from the earliest convenience increasing the efficiency of post-parsing. As such,

[3] Otherwise cross-partition references are flagged as errors.

their purpose will be discussed in the next section.

Thread-safety is required for all operations in each `Runnable` as explained previously. As helpfully pointed out by EMF's Project Lead Ed Merks, there is no contention when loading resources from different resource sets [38]. Similarly, shared access of the in-memory representation of the original file poses no risk simultaneous reading is thread-safe. When a resource's loading completes it is added to a `ConcurrentHashMap parsedResources` with the partition's original index as its key. This ensures that *a.* there are no issues related to the concurrent adding of many partitions to the collection, and *b.* the original order of partitions is maintained. The key of each loaded resource is also added to a (thread-safe) `BlockingQueue`, `parsedKeys`, used for notifying the next stage in the pipeline as described later. Our final synchronisation concern is the termination of each worker-thread, which is performed using `ExecutorService.shutdown()` and `ExecutorService.awaitTermination(..)` as soon as `Runnable`s are added to the `ExecutorService`'s queue for every partition.

4.3 Post-Parsing

As this step is pipelined with the previous step, it begins executing when the first (in order) resource is loaded. More precisely, its execution commences when it is notified by the `parsedKeys BlockingQueue` that a resource has been parsed, and the resource has the key 0. If any other resources are loaded before the first, their keys are stored locally and the thread sleeps until it is notified once again by the `BlockingQueue`. This mechanism repeats until all partitions are loaded and joined, when a final method call corrects the lack of forward and cross-partition references.

Phase One: Sub-Tree Reconciliation

For our master `ParallelXMIResource` to hold the contents of the complete model every object of each `SubXMIResource` loaded by the previous pipeline stage needs to be reassigned to the maser resource, a process outlined in Algorithm 3. This phase takes advantage of EMF's tree-returning `Resource.getAllContents`, as both the master and sub-resource are traversed depth-first[4] until its contents tree points to the `EObject` representing the parent of the partition's content. This object's containments in the sub-resource are each accessed separately as

[4] The sub-resource and master are traversed in the same number of steps, but the path walked for the master must consider the index for each entry in `ancestorIndices`.

EObjects with their own containments by breadth-first traversal of the content tree. This is performed by repeating Resource.getAllContents-.next() and Resource.getAllContents.prune() whilever Resource.getAllContents.hasNext() returns true. The results of each next() operation are stored in an intermediate list, before being added to the master parent node's contents[5].

Rather pleasantly, a restriction exists in EMF dictating that objects can only belong to one resource at a time [29, p. 451]; assigning an object to another resource simply destroys the previous link and forms another without any content being copied. Although the process of locating the splicing position is reasonably complex it performs well both in speed and memory utilisation; links are broken and reformed exactly once for every object and no duplication of any kind arises. A side effect of this, however, results in the extra operations performed in the previous step. Each Resource stores mappings between the ID and referent EObject for each XMI element that contains a xmi:id attribute, namely eObjectToIDMap and its inverse idToEObjectMap, useful for finding the target of EReferences.

These mappings are only stored on an object's original resource, so moving every object in each sub-resource to the master presents a problem: without any further processing any and all references to object IDs are 'forgotten' by EMF resulting in XMI that closely resembles ordinary XML as shown in Figure 4.3. To ensure the integrity of such references the master resource's maps must contain the sum of the content of each sub-resource's maps. This is an expensive operation; it is not only inherently sequential due to the updating of state variables, but also copying large maps of potentially millions of entries is generally slow. Although introducing task parallelism with two Executors.newSingleThreadExecutor())s and Runnables that copy each map in a separate thread helps the issue, given enough elements with ID attributes, I can see this very easily becoming the dominating performance bottleneck of P4EMF. Fortunately EMF also supports referencing by relative paths, and on models using this referencing scheme this issue is not encountered.

Before starting the next phase of post-parsing there is a special case that needs to be considered. EMF considers generic root <xmi:XMI> elements to have absolutely no content per se. When joining XMI documents that begin with this element the partition containing only the root node is simply ignored, as is the first element of XMIPartition.ancestorIndices. This is perfectly legal as while XML documents must be well-formed, EMF models can have several root elements and indeed serialising

[5] By calling master.eSet(object.eContainingFeature(), object) if the sub-resource's node's feature cannot have several values, or master.eGet(object.eContainingFeature()).add(object) otherwise. As I describe in Section 6.1, this process becomes the limiting bottleneck on my prototype's performance.

```
<?xml version="1.0" encoding="ASCII"?>
<university:University xmlns:university="university" xmi:id="u1" name="University of York">
    <lecturers xmi:id="l1" name="lecturer1"/>
    <lecturers xmi:id="l2" name="lecturer2"/>
</university:University>

<?xml version="1.0" encoding="ASCII"?>
<university:University xmlns:university="university" xmi:id="u1" name="University of York">
    <subjects xmi:id="s1" name="subject1" lecturer="l1" />
    <subjects xmi:id="s2" name="subject2" lecturer="l2" />
</university:University>
```

```
<?xml version="1.0" encoding="ASCII"?>
<university:University xmlns:university="university" name="University of York">
    <lecturers name="lecturer1"/>
    <lecturers name="lecturer2"/>
    <subjects name="subject1" />
    <subjects name="subject2" />
</university:University>
```

Figure 4.3: Top: two valid partitions of XMI with references and `xmi:id` attributes. Bottom: serialisation of the model formed by loading and joining each partition without the duplication of each resource's ID-Object map.

such a model results in XMI with a reconstructed `<xmi:XMI>` root element.

Phase Two: Cross-Partition Reference Handling

The very last step of P4EMF is identical to the final step of the native loading process: forward reference handling. Not only is this step of importance for creating the necessary links between cross-partition references, but also required to ensure the initial XMI document is indeed valid. The precondition of executing `XMLHandler.handleForwardReferences(..)` is the existence of the state variables `sameDocumentProxies`, `forwardManyReferences`, and `forwardSingleReferences` that describe the forward references identified during its parse. Rather than another scan of the loaded resource's `contents`, the relevant subset of the state of each loaded partition's `XMLHandler` can be stored in the resource at very little cost. Once all partitions are loaded these variables be joined to construct the state of our `ParallelXMIResourceImpl`'s `XMLHandler` and the `handleForwardReferences(..)` method can be executed with the assurance that the final model will have undergone the same checks as if it had been parsed natively.

The correctness of P4EMF was tested by executing `Resource.save(null)` after each resource was loaded by P4EMF and using Diffchecker [4] to verify that its XMI serialisation did not change. Distinct models are represented by distinct serialisations, and since the serialisations generated after sequential and P4EMF loading of the same XMI file are indistinguishable it follows that they represent identical in-memory models.

5 Analysis

This section aims to examine the performance of my prototype implementation of P4EMF in terms of execution time and memory footprint when compared with the sequential loading process. All experiments were performed on the University of York's high performance *Viking Cluster* [5], with access to 16 Intel Xeon Gold cores and 20GB of memory.

Section 5.1 takes a close look at the performance of my pre-parsing algorithms to unearth any potential consequences on the performance of P4EMF as a whole by recording the mean[1] execution time of each pre-parsing phase when loading the best and worst-case models suggested in Section 3.1, and a similarly sized real data set, five times. Section 5.2 analyses the overall performance of my P4EMF prototype by loading XMI models of varying sizes with both P4EMF (varying the number of threads) and the sequential loading process, and recording the time taken to complete. Each experiment was repeated ten times and of these results the largest two were removed to allow for anomalies (for example unrelated background tasks or OS operations) with the mean of the remaining eight results plotted. Finally, Section 5.3 considers the additional memory requirements of P4EMF in comparison with the existing loading process, performed by using VisualVM [39] to identify the peak usage of the heap after loading the same models as before, using the mean of five repeats.

5.1 Pre-Parsing Overhead Analysis

5.1.1 Best Case Performance

The best case performance of my pre-parsing algorithms occurs with models conforming to the structure in Figure B.7 (for example, Listing C.2). Focusing on Phase 2 of the results in Figure 5.1 (the first and third phases are determined, at least somewhat, by the content of the file) shows us the best-case sample outperforming the real data set by a factor of 1.6. As each of the m first level children of the root form own partition, only $m + 1$ operations are required, forming the minimum number of partitions. The

[1] The mean of each experiment is used as the deviance between results is low; taking the median instead would have no significant effect on my results.

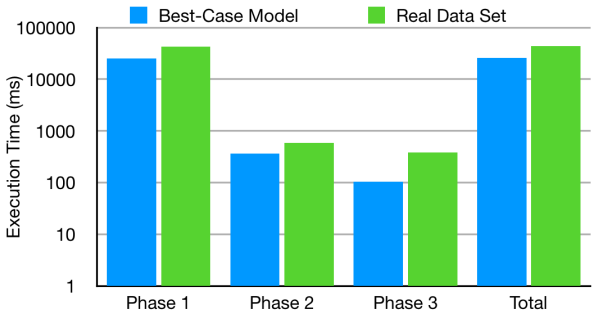

Figure 5.1: Chart showing execution times of pre-parsing phases for both best-case sample data and a real data set. n=200,000; m=4.

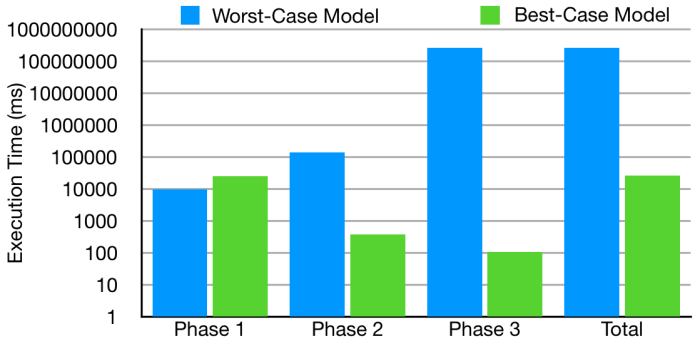

Figure 5.2: Chart showing execution times of pre-parsing phases for both worst and best-case sample data. n=200,000; m=4.

final phase of pre-parsing also performs well; only one upwards traversal is required before the root of the tree is found for each partition (m operations).

5.1.2 Worst Case Performance Model

Conversely, my pre-parsing step performs at its worst with models similar to B.6 (for example Listing C.3), where it becomes the clear bottleneck of P4EMF. This is partly due to the $2n - x$ operations required to partition such a model's document tree as described in Section 3.1, however the performance of the final pre-parsing phase is much more responsible requiring string manipulation operations repeated close to $\frac{(n-\frac{n}{m})(n-\frac{n}{m}+1)}{2}$ times. To add some context consider the final pre-parsing phase for best and worst-case XMI samples of one hundred elements (n=100) to be parsed with four threads (m=4): the best-case requires no more than four steps through the tree to copy each partition's sole parent element; the worst-case model requires 2850. For 200,000 elements, this figure is roughly ten million in the worst-case, yet still four in the best. These issues are both clearly shown in Figure 5.2 where the second phase is roughly 400 times slower with the worst-case model, and phase three is more than two million time slower.

5.2 Typical Use Analysis

My initial measurements were disappointing and showed P4EMF significantly underperforming against the existing process with any number of threads. This initial discrepancy was caused by two of Java's internal features: garbage collection and the use of Tiered Compilation[2]. To reduce the influence of the JVM's frequent and intensive garbage collection process `System.gc()` was run before and after each iteration, manually triggering a garbage collection cycle to reducing the chances of it being run again during my program's execution. To mitigate the effects of JIT and tiered compilation every experiment began by loading a different model several times using the sequential process until the required classes were compiled.

After these modifications, the results of my experiments (blue line shown in Figures 5.3, 5.4, and 5.5) matched my expectations: the use of one loading thread shows poor performance followed by a significant speed improvement with more threads demonstrating that P4EMF loading can be up to 108% faster than the sequential loading process. As can be seen in each of the charts there is a clear bottleneck limiting the benefits of data-parallelism to a maximum of around seven threads in all experiments. After investigation this bottleneck was found to be the task of copying the large maps as predicted in the Section 4.3, caused by each element of my sample data, as output by EMF's default serialiser, containing an `xmi:id` attribute regardless of if the element is ever referenced or not, hence bloating the maps with unnecessary entries.

EMF can handle object references not only in terms of IDs but also by relative path URIs as seen in Listing C.4. The loading of models containing few references performs much better with the latter method and, as expected, so does P4EMF. The absence of the need to copy large maps removes the previous bottleneck and allows a significant improvement in performance, shown as the red lines in my result charts. While execution time was cut by a large proportion in all experiments this effect was also seen by the sequential process. Consequently, while using path-based object referencing can load models much faster than with ID-based referencing, the P4EMF's speedup improved only slightly, performing 120% faster than the sequential loading process. After more investigation I identified the new bottleneck as the sub-resource joining process; a limitation of my architecture.

[2] Simply put, the tiered execution of Java bytecode on the JVM improves performance by compiling only the most frequently run blocks of code and interpreting the rest [40].

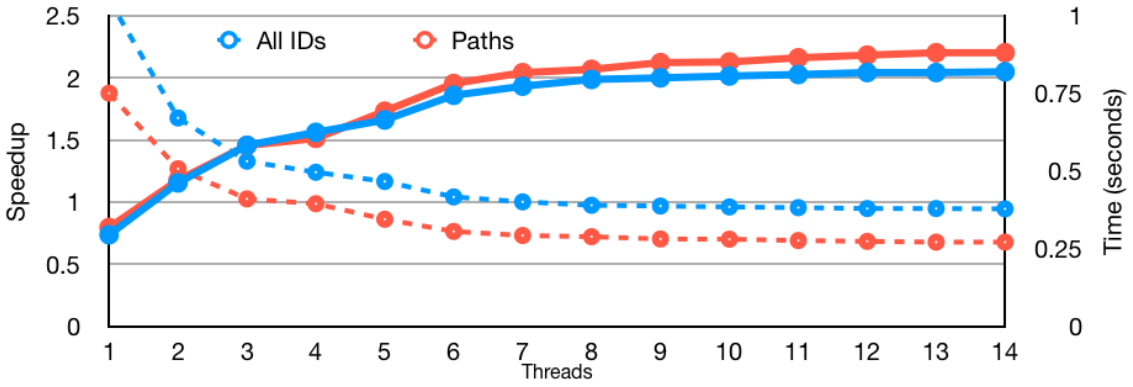

Figure 5.3: A chart showing the measured speedup of loading sample data set 1 (model of ~200,000 elements) with P4EMF compared with EMF's native loading process (solid) for both ID and path-based object referencing, and their mean absolute execution times (dashed).

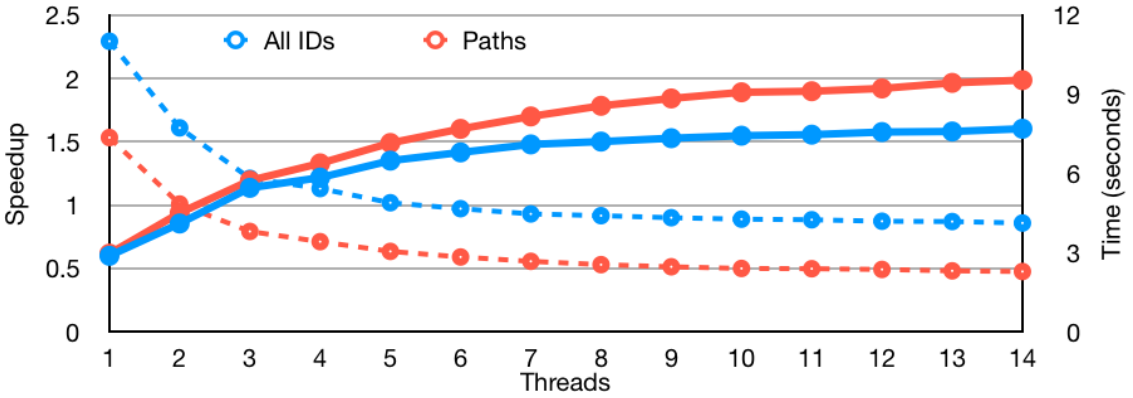

Figure 5.4: A chart showing the measured speedup of loading sample data set 2 (model of ~2,000,000 elements) with P4EMF compared with EMF's native loading process (solid) for both ID and path-based object referencing, and their mean absolute execution times (dashed).

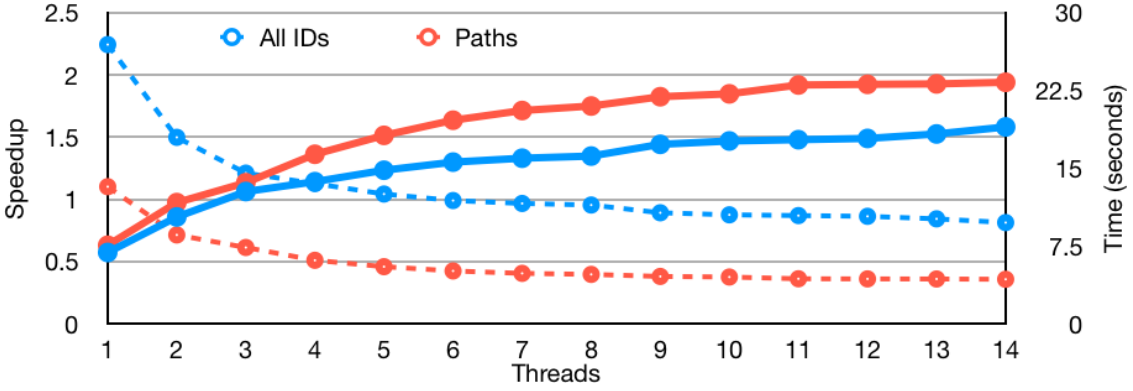

Figure 5.5: A chart showing the measured speedup of loading sample data set 3 (model of ~4,500,000 elements) with P4EMF compared with EMF's native loading process (solid) for both ID and path-based object referencing, and their mean absolute execution times (dashed).

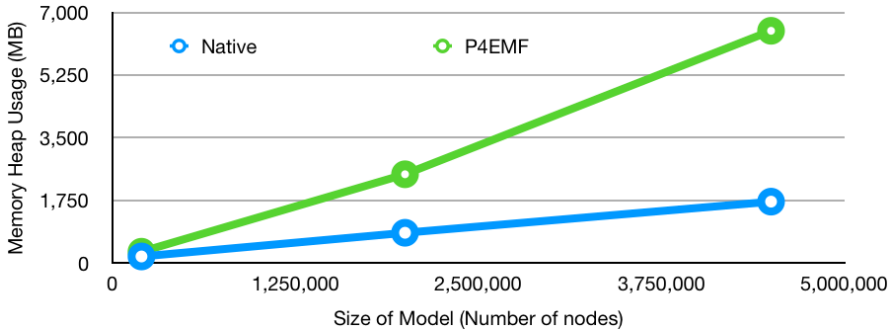

Figure 5.6: A chart showing the average peak memory usage of the native and P4EMF loading processes on models of varying sizes.

5.3 Memory Use Analysis

The results from this experiment should be taken as a possible indication and a general trend. Identifying the memory usage of either the sequential loading process or P4EMF is imprecise and dependant on many factors including the model's size, structure, and the number of threads used to load it. As Figure 5.6 shows, heap[3] memory usage of P4EMF is significantly larger than the native loading process when loading the same model. More samples are required to predict its growth with respect to the size of models, this shows that around 6.5 GB of heap memory was required to load the largest of the three models. This should be of little concern for high performance computer networks, although consumer equipment may be unable to use P4EMF due to memory restrictions while being able to support the execution of the sequential loading process. These results do not come as a surprise for me, as it can be expected that generating, storing, and processing chunks of the original file into many new resources will have a considerable effect on the memory of a process that previously read only from a stream of the source.

5.3.1 Potential Speedup

When considering parallelisation of a sequential program, it is wise to consider what the potential benefit of the parallelisation of a system could bring. Amdahl's Law (or *argument*) can be used to calculate the theoretical maximum speedup by comparing the proportion of processing that must remain sequential with the processing that can be made parallel [32] (specifically data-parallel).

Figure 5.7 shows the proportion of execution time for each execution block,

[3]There are two forms of JVM memory: the heap stores the runtime class instances and arrays, and changes in size depending on how much memory is required; non-heap memory stores constructor, method and class data created at the JVM startup [41].

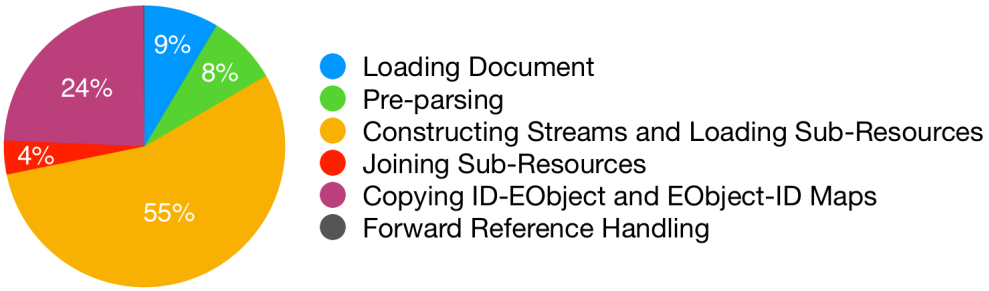

Figure 5.7: Proportion of processor time allocated to each task. Note the insignificance of forward reference handling, which accounts for only 0.2% of the overall execution.

as measured by loading a sample data set (set2) without the use of pipelining and with only a single loading thread. Pipelining cannot be represented in this calculation, but as (with enough threads) it always improves performance it will surely not reduce the result. Assuming execution times of the sequentially running blocks will not fluctuate with varying numbers of loading threads[4] and that my pre-partitioning step forms correctly balanced partitions such that doubling the size of the loading thread pool increases performance of the data-parallel section by a two-fold, the theoretical maximum speedup of P4EMF as defined by Amdahl's Law is calculated as[5]:

$$S = \frac{1}{1-p} = \frac{1}{1-0.55} = 2.222$$

A more realistic calculation would consider task-parallelism, and assume that no time is spent waiting for the task-parallel Map-Copying stage to complete. Indeed I have shown this is a feasible and realistic assumption to make if path-based object referencing is used.

$$S = \frac{1}{1-p} = \frac{1}{1-0.73} = 3.704$$

This result is far higher than what was achieved in any experiment. This would suggest that my assumptions are not satisfied, chiefly that my pre-parsing step fails to generate genuinely balanced partitions. This is true, and is something which should be of focus in further work.

[4] This is not strictly the case. Increasing the number of threads, and hence partitions, resources, etc. will slightly increase execution times, although in practice this should be negligible.

[5] As an aside, it is interesting to note how my experiment results (especially in Figure 5.3) seem to converge on a value very close to 2.2. I believe that the convergence to this particular value is purely coincidence due to the calculation's described inaccuracies.

6 Conclusion and Further Work

6.1 Conclusion

This report introduces a promising new approach to parsing large XMI files, and provides a prototype implementation for the Eclipse Modelling Framework that can allow models to be loaded into memory more than two times faster than EMF's native loading process. Data parallelism really is the key to achieving substantial speed gains as demonstrated in the previous research by Lu et al. [13], but as my use of Amdahl's law suggests without any other improvements this is physically limited to a speedup of just over 100%. Introducing hybrid parallelism with a two stage pipeline slightly improves this figure, but the loading process is still largely dominated by the cost of sequentially joining the loaded sub-resources back together. Although the post-parsing tasks remain the limiting factor for P4EMF, delegating certain implementation specific tasks to background processes via task-parallelism is a novel solution to reduce its impact. Gordon may have rightly said that task-parallelism "is not sufficient as the only source of parallelism" [27], but it can sure be useful.

The hybrid data and pipeline-parallel solution to XML parsing formed by Pan et al. was demonstrated to load documents up to four times faster than existing SAX parsers [3]. I believe the reason I was unable to get speed improvements of a similar level is that unlike a lightweight XML parser, my prototype is still heavily relent on the internal processes provided by EMF. In the case of the sequential joining bottleneck the main restriction is EMF's handling of shifting the contents of one resource to another which, while not copying anything, results in a number of internal methods being run for *every* object in each resource. For small models this has less of an effect, as seen by the larger speedup in Figure 5.3 as opposed to Figure 5.5, but for larger models this process becomes more dominating. This report demonstrates that a large performance improvement is still possible in these circumstances, and I would be inclined to bet that a *much* larger improvement is possible if a new class of EMF resource is constructed; one that is developed with the intention of being constructed from many sub-resources.

My memory utilisation figures should be taken with a pinch of salt; they represent possible results that vary and will likely be different if reproduction of

the experiment is attempted. What they do show, as previously mentioned, is that P4EMF loading requires far more heap-memory than used the native process. I believe P4EMF could have significant impact on the loading times of huge models (think gigabytes), although without enough memory it will not be possible to use. Similarly, Figure B.10 demonstrates how speed improvements will only be present on machines with five or more available processors or cores[1]. Both of these requirements isolate the consumer PC market and limit the applicability of my implementation to high performance computing applications.

All in all, I am very happy with the performance benefits that P4EMF can produce and, if nothing else, this report proves the hypothesis that parallelisation can be used to build a faster and more scalable XMI parser. My personal development over the course of this project has progressed in leaps and bounds from the initial learning curve with not only EMF and EMF's existing model loading process, but also with parallelism as a whole and, indeed, Java. I feel confident that I have made the best of my ability and chance to contribute to cutting-edge research, and am sure that it will be of use to the active modelling community.

6.2 Further Work

While my results are promising, there are many aspects that require further work. My pre-parsing algorithm is quick enough to have little impact on performance for most cases, although as highlighted in my analysis there are cases where it becomes the bottleneck. Work should therefore be carried out to remove this issue, which may involve altering the slicing algorithm in the second phase to one which does not produce more partitions than the number of threads used to load them. Work is also desired to identify a better heuristic to use for the weight of each node that can be efficiently calculated and, given an optimal partitioning algorithm, will ensure a more optimal division of the XMI tree in terms of the time required to load each partition.

In addition to these suggestions I believe more work is required to find further optimisations to P4EMF, specifically concerning options and caching. My implementation of P4EMF internally uses only one EMF loading option, `OPTION_DEFER_IDREF_RESOLUTION`, to make certain that no time is spent trying to locate cross-partition references during the initial loading of each chunk. An avenue of further work consists of identifying other loading options that can improve the performance of the parallel loading of XMI chunks, particularly when compared with the native loading process

[1] While this is true in general, assuming all referencing uses relative path URIs instead of object IDs, only three cores are required to observe noticeable speed improvements.

with the same options. Similarly, it may be possible to cache the results of pre-parsing, and other data, so that it can be recovered for subsequent loads of the same model, improving loading performance.

As noted in my conclusion while there may be a workaround to remove the bottleneck of ID-EObject map copying there is no workaround to removing or reducing the limiting factor caused by the slow process of joining sub-resources together. Further work ought to be carried out to examine this internal process more deeply and identify a more efficient method of, or even parallelise, the process of moving content of sub-resources to fixed positions within a master resource. If the proportion of overall execution that is taken up by joining resources can be reduced even slightly I believe the improvements will be fairly substantial, due to Amdahl's Law.

Finally, I recommend that a production implementation of P4EMF should be built and bundled with EMF releases. While concern should be directed to P4EMF's performance on low and mid-range machines, with the impressive improvements documented in my analysis I see no reason why a version of my implementation should not be included with EMF once the above issues are dealt with appropriately. Unfortunately, this is not an easy task and intensive testing, compatibility fixes, and other improvements will be required to turn my prototype into a product.

A Algorithms

```
let childIndexStack = new Stack();
let currentEndNode = null;
let currentNode = null;
Function GetPartitions():
    let numberOfThreads = getNumberOfParsingThreads;
    let partitions = new List();
    let equalPartitionSize = rootNode.recursiveChildCount/numberOfThreads;
    currentNode = rootNode;
    while partitioning not complete do
        partitions.add(getPartitionOfMaxSize(equalPartitionSize));
    end
    return partitions
Function GetPartitionOfMaxSize(maxChunkSize):
    let currentPartitionSize = 0;
    if currentNode.recursiveChildCount + currentPartitionSize + 1 <= maxPartitionSize then
        currentPartitionSize = currentPartitionSize + currentNode.recursiveChildCount + 1;
        if currentNode has no parent then
            return new XMIPartition containing the entire document;
        end
        if currentNode.parent has any more children not in a partition then
            let currentEndNode = currentNode;
            for let i = childIndexStack.peek() + 1 to number of children of currentNode.parent do
                let nextSibling = currentNode.parent.children.get(i);
                if nextSibling can also fit into the partition then
                    currentPartitionSize = currentPartitionSize +
                      nextSibling.recursiveChildCount + 1;
                    currentEndNode = nextSibling;
                    if nextSibling is currentNode.parent's final child then
                        let currentPartition = new XMIPartition containing elements from
                          currentNode to currentEndNode;
                        currentNode = get parent's next sibling (recursive);
                        return currentPartition;
                    end
                else
                    let currentPartition = new XMIPartition containing elements from
                      currentNode to currentEndNode;
                    currentNode = nextSibling;
                    childIndexStack.pop();
                    childIndexStack.push(i);
                    return currentPartition;
                end
            end
        else
            // currentNode fits but has no more siblings
            let currentPartition = new XMIPartition containing only currentNode;
            currentNode = get parent's next sibling (recursive);
            return currentPartition;
        end
    else
        // currentNode is too large to fit into partition
        let currentPartition = new XMIPartition containing only currentNode;
        currentPartition.isEmptyParentNode = TRUE;
        childIndexStack.push(0);
        currentNode = currentNode's first child;
        return currentPartition;
    end
```

Algorithm 1: Algorithm used to split an XMI tree into a number of partitions with a ceiling on the number of nodes in each.

```
Procedure FixPartition(partition):
    let header, footer;
    let finalNode = partition.endNode;
    if finalNode line type = Open Tag then
        appendToFooter(concat("</", getElementNameOfNode(finalNode),
          ">"));
        let localParentNode = finalNode.parent;
        while localParentNode != partition.parentNode do
            appendToFooter(concat("</",
              getElementNameOfNode(localParentNode), ">"));
            localParentNode = localParentNode.parent;
        end
    end
    let parentClosingTags = new Queue();
    let parentNode = partition.parent;
    while parentNode != null do
        insertToFrontOfHeader(getLineOf(parentNode));
        appendToFooter(concat("</",
          getElementNameOfNode(parentNode), ">"));
        partition.headerLineCount = partition.headerLineCount + 1;
        parentNode = parentNode.parent;
    end
```

Algorithm 2: Algorithm that constructs strings to make an XMI chunk valid when added to its beginning and end.

```
List<String> chunk = new ArrayList<String>();
chunk.add(partition.headerLines);

if (partition.isEmptyparentNode) {
  chunk.add(document.get(partition.startNode.startLine - 1) +
      '\n');
}
else {
  int endLineNumber = partition.endLineNumber == -1 ?
     partition.endNode.startLine: partition.endLineNumber; //
      In case final node is empty.
  chunk.addAll(document.subList(partition.startNode.startLine
      - 1, endLineNumber));
}
chunk.add(partition.footerLines);

ByteArrayOutputStream baos = new ByteArrayOutputStream();
for (String line : chunk) baos.write(line.getBytes());

InputStream partitionedXMIStream = new ByteArrayInputStream(
   baos.toByteArray());
```

Listing A.1: Java snippet of the process that builds an InputStream from the in-memory ArrayList of the original document.

```
Procedure JoinSubResources():
    wait for parsedResources[0] to become available;
    if document does not begin with <xmi:XMI.. > element then
        add parsedResources[0].contents[0] to master.contents;
    end
    for let index = 1 to parsedResources.size do
        wait for parsedResources[index] to become available
         insertContentsIntoMaster(resource);
    end
Procedure InsertContentsIntoMaster(resource):
    for let index = 0 to resource.ancestorIndices.size-1 do
        // Traverse to correct position in sub-resource.
        if index = 0 AND document begins with <xmi:XMI.. > element then
            continue
        else
            resource.getAllContents().next();
        end
    end
    // Get all first level children.
    let chunkData = new List();
    while resource.getAllContents().hasNext() do
        chunkData.add(resourceContents.next());
        resourceContents.prune();
    end
    // Traverse to correct position in master resource.
    let parentNode = null;
    if document does not begin with <xmi:XMI.. > element then
        parentNode = master.contents[0][0];
    end
    foreach index, v ∈ resource.ancestorIndices do
        if index = 0 AND document begins with <xmi:XMI.. > element then
            parentNode = master.contents[v];
        else
            parentNode = parentNode.eContents[v];
        end
    end
    foreach eObject ∈ chunkData do
        if parentNode != null AND eObject.eContainingFeature().isMany()
         then
            parentNode.eGet(eObject.eContainingFeature()).add(eObject);
        else if parentNode != null then
            parentNode.eSet(eObject.eContainingFeature(), eObject);
        else
            master.contents.add(eObject);
        end
    end
```

Algorithm 3: Algorithm used to join loaded XMI partitions together to recreate the same structure initially provided.

B Diagrams

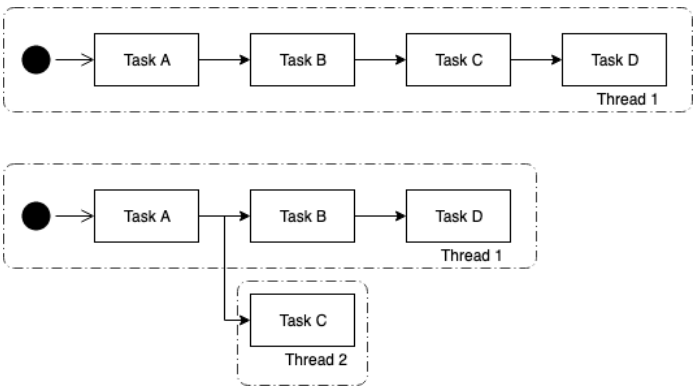

Figure B.1: Abstract example of the task parallelisation of a sequential program. Task C does not modify Task D's context.

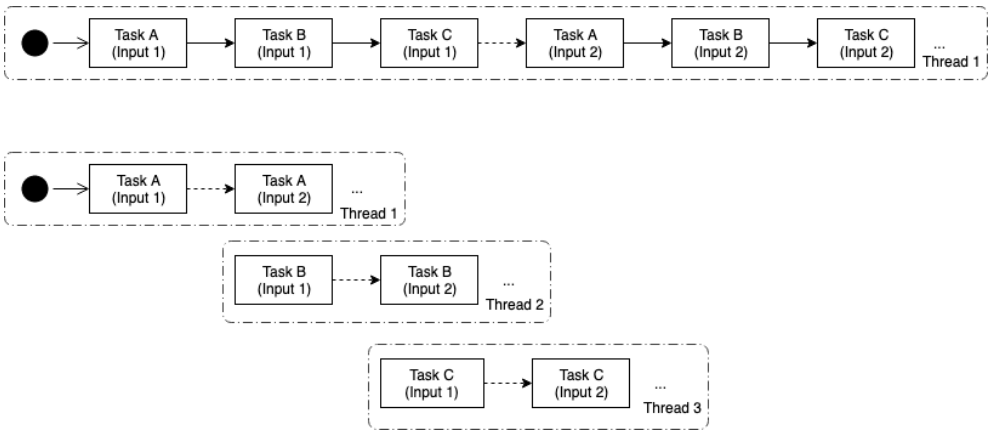

Figure B.2: Abstract example of the three-stage pipeline parallelisation of a sequential program which processes inputs with three tasks.

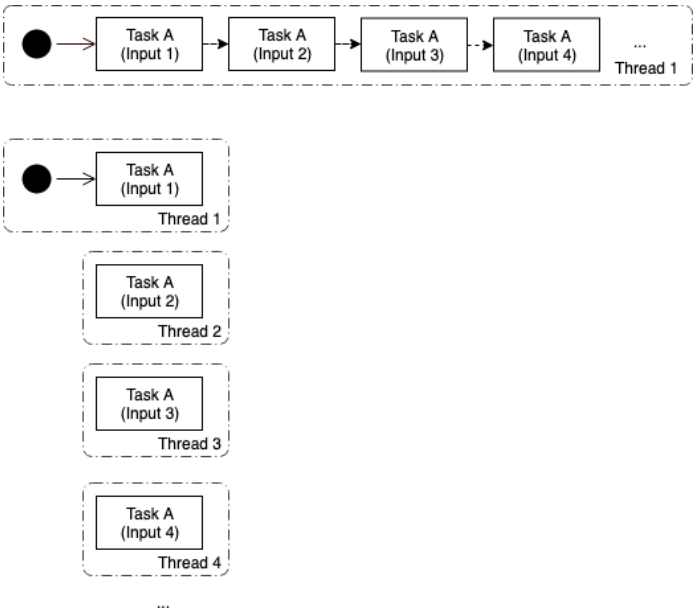

Figure B.3: Abstract example of the data parallelisation of a sequential program that processes each input with a single task (assuming all inputs are initially available).

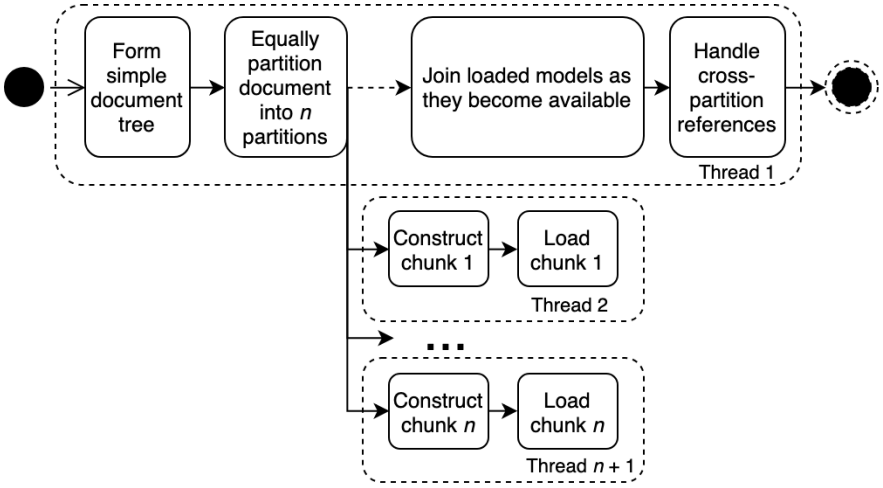

Figure B.4: A simple diagram showing the parallelisation of blocks in the design of P4EMF.

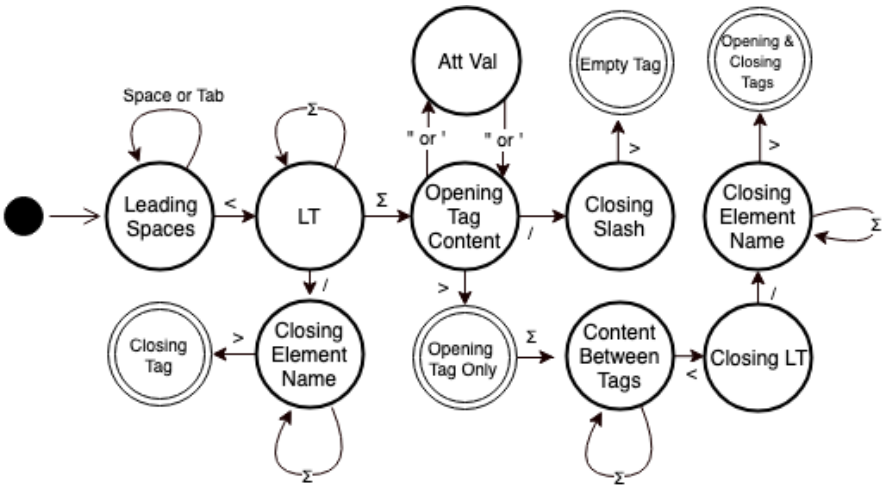

Figure B.5: A detailed diagram of the Finite State Machine that classifies a line of XMI provided my assumptions stand. Note that Σ is used to denote all valid XML characters excluding $<$, $>$, ", ', and $/$.

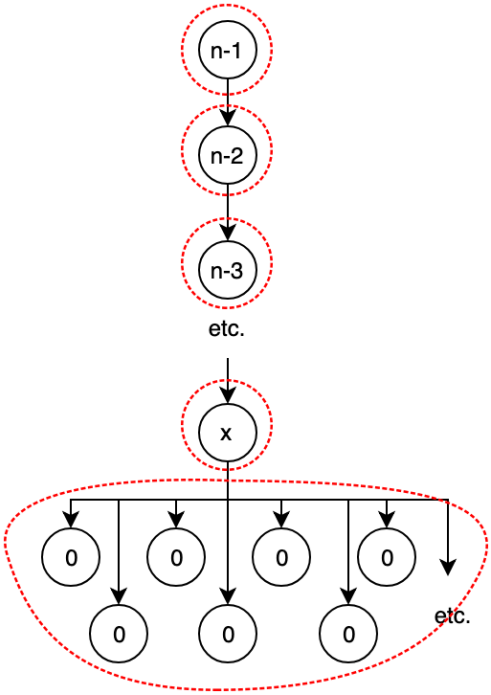

Figure B.6: The tree that exemplifies the worst case behaviour of my partitioning algorithm, displaying the weight of each node; x is the maximum weight allowed in each partition; n is the total number of nodes.

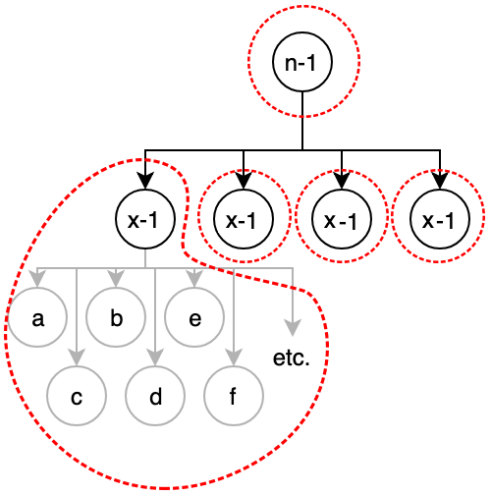

Figure B.7: A tree that exemplifies the best case behaviour of my partitioning algorithm displaying the weight of each node where $m = 4$; x is the maximum weight allowed in each partition; n is the total number of nodes and nodes labelled a, b, c etc. can have any structure whatsoever.

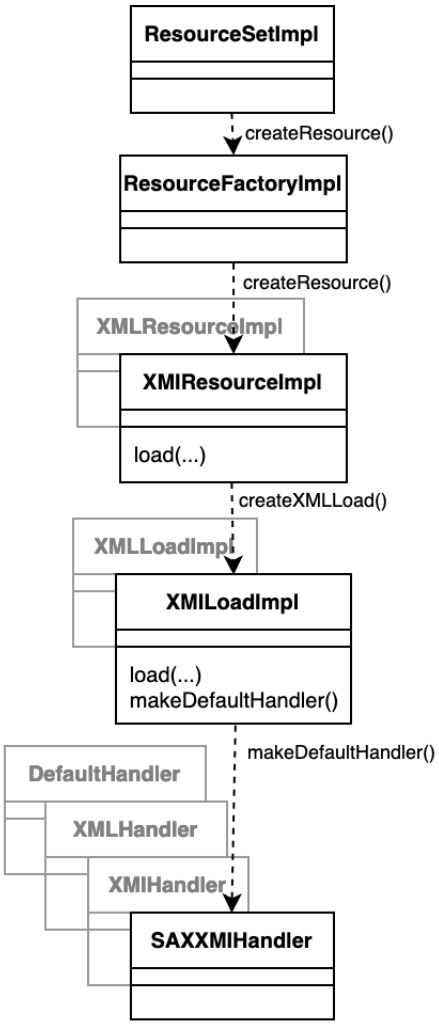

Figure B.8: EMF SAX Parser Structure, extended from [12].

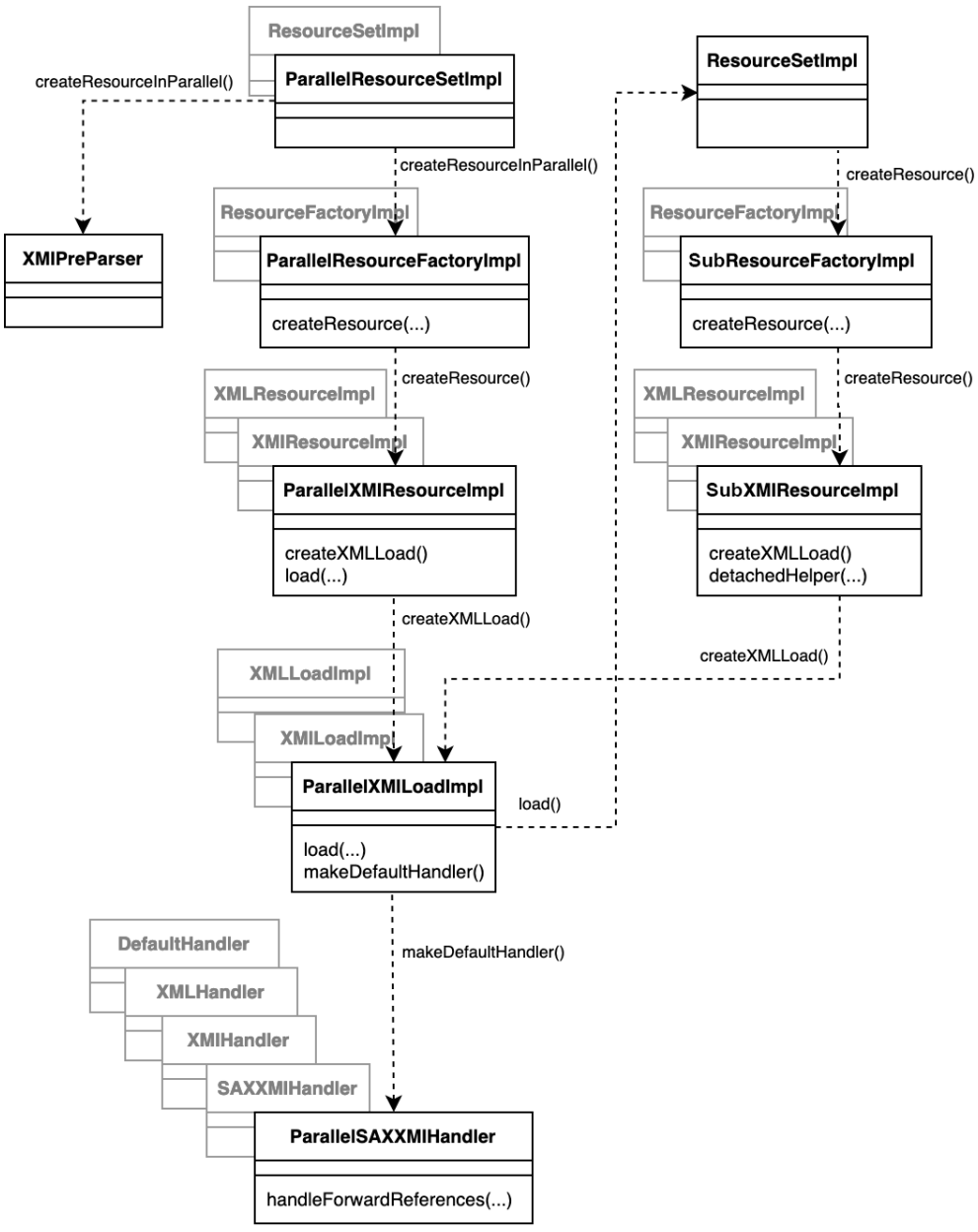

Figure B.9: P4EMF Structure, showing overridden methods and inherited classes.

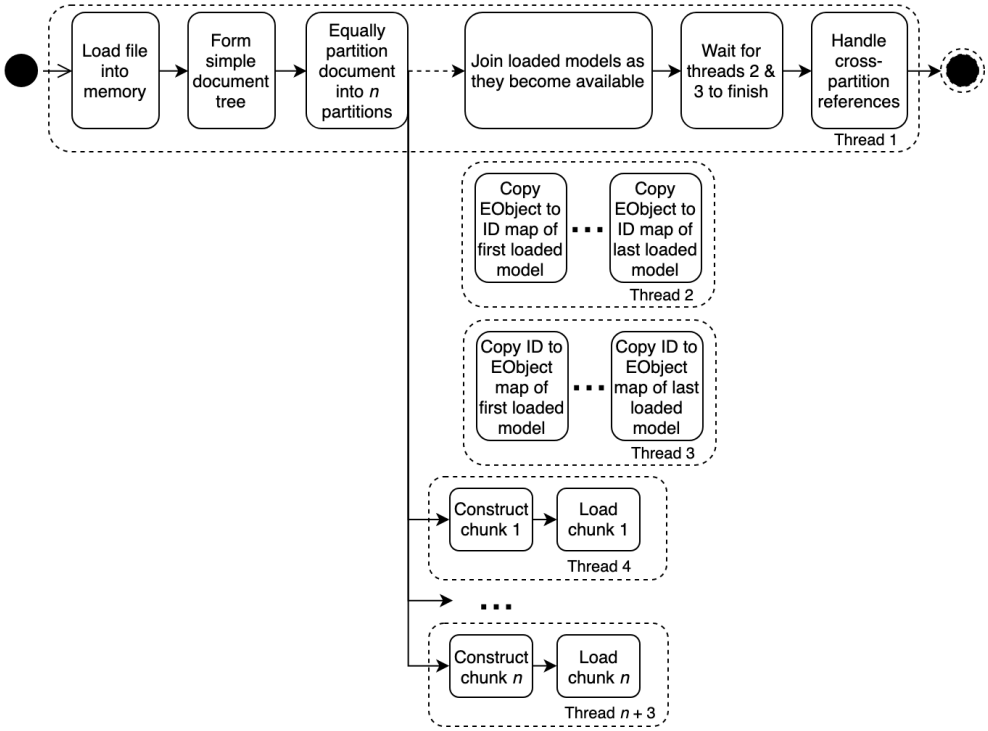

Figure B.10: A simple diagram showing the parallelisation of blocks in the implementation of P4EMF.

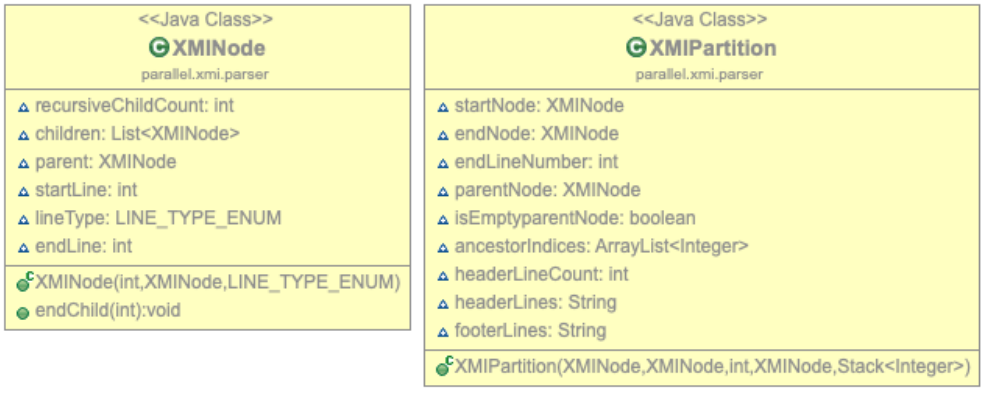

Figure B.11: UML Class Diagrams of `XMINode` and `XMIPartition`.

C XMI Samples

```
<ecore:EPackage xmi:version="2.0"
    xmlns:xmi="http://www.omg.org/XMI" xmlns:xsi="http://www.
       w3.org/2001/XMLSchema-instance"
    xmlns:ecore="http://www.eclipse.org/emf/2002/Ecore" name=
       "university" nsURI="university" nsPrefix="university">
    <eClassifiers xsi:type="ecore:EClass" name="University">
        <eStructuralFeatures xsi:type="ecore:EAttribute" name
           ="name" eType="ecore:EDataType http://www.eclipse.
           org/emf/2002/Ecore#//EString"/>
        <eStructuralFeatures xsi:type="ecore:EReference" name
           ="lecturers" upperBound="-1" eType="#//Lecturer"
           containment="true"/>
        <eStructuralFeatures xsi:type="ecore:EReference" name
           ="subjects" upperBound="-1" eType="#//Subject"
           containment="true"/>
    </eClassifiers>
    <eClassifiers xsi:type="ecore:EClass" name="Lecturer">
        <eStructuralFeatures xsi:type="ecore:EAttribute" name
           ="name" eType="ecore:EDataType http://www.eclipse.
           org/emf/2002/Ecore#//EString"/>
    </eClassifiers>
    <eClassifiers xsi:type="ecore:EClass" name="Subject">
        <eStructuralFeatures xsi:type="ecore:EAttribute" name
           ="name" lowerBound="1" eType="ecore:EDataType
           http://www.eclipse.org/emf/2002/Ecore#//EString"/>
        <eStructuralFeatures xsi:type="ecore:EReference" name
           ="lecturer" lowerBound="1" eType="#//Lecturer"/>
    </eClassifiers>
</ecore:EPackage>
```

Listing C.1: Sample XMI metamodel

```
<?xml version="1.0" encoding="ASCII"?>
<xmi:XMI>
  <university:University name="university">
    <lecturers name="Lecturer11" />
    <lecturers name="Lecturer12" />
  </university:University>
  <university:University name="university">
    <lecturers name="Lecturer21" />
    <lecturers name="Lecturer22" />
  </university:University>
</xmi:XMI>
```

Listing C.2: Sample of the best-case XMI structure if n=7 and m=2.

```
<?xml version="1.0" encoding="ASCII"?>
<xmi:XMI>
  <xmi:XMI>
    <xmi:XMI> // n-(n/m) elements deep.
      <university:University name="university" />
      <university:University name="university" />
      <university:University name="university" />
    </xmi:XMI>
  </xmi:XMI>
</xmi:XMI>
```

Listing C.3: Sample of the worst-case XMI structure if n=6 and m=2.

```
<universities:Universities>
  <university location="York" chancellor="c1">
    <chancellor xmi:id="c1" name="Malcolm Grant" />
  </university>
</universities:Universities>

<universities:Universities>
  <university location="York" chancellor="//@university.0/
     @chancellor.0">
    <chancellor name="Malcolm Grant" />
  </university>
</universities:Universities>
```

Listing C.4: Comparison of ID-based and path-based object referencing in serialised EMF models.

Bibliography

[1] *A Look Back at Single-Threaded CPU Performance*, https://preshing.com/20120208/a-look-back-at-single-threaded-cpu-performance/, Accessed: 2019-02-12, 2012.

[2] H. SUTTER, 'The free lunch is over: A fundamental turn toward concurrency in software', *Dr. Dobb's Journal*, vol. 30, Jan. 2005.

[3] Y. Pan, Y. Zhang and K. Chiu, 'Hybrid parallelism for xml sax parsing', in *2008 IEEE International Conference on Web Services*, Sep. 2008, pp. 505–512. DOI: 10.1109/ICWS.2008.107.

[4] *Diffchecker*, https://www.diffchecker.com/, Accessed: 2019-04-27.

[5] University of York, *Viking Cluster*. [Online]. Available: https://www.york.ac.uk/it-services/research-computing/viking-cluster/ (Accessed: 04/04/2019).

[6] *XML 1.0 Origin and Goals*, https://www.w3.org/TR/REC-xml/#sec-origin-goals, Accessed: 2019-02-20, Nov. 2008.

[7] *XML for the uninitiated*, https://support.office.com/en-us/article/xml-for-the-uninitiated-a87d234d-4c2e-4409-9cbc-45e4eb857d44, Accessed: 2019-03-02.

[8] *XML Format Files (SQL Server)*, https://docs.microsoft.com/en-us/sql/relational-databases/import-export/xml-format-files-sql-server, Accessed: 2019-02-07, 11th Jan. 2019.

[9] *About the xml metadata interchange specification*, https://www.omg.org/spec/XMI/About-XMI/, Accessed: 2019-02-07, 2015.

[10] *XML Metadata Interchange: An OMG SMIF Proposal*, http://xml.coverpages.org/xmi-ibm980612.html, Accessed: 2019-02-07, 1998.

[11] J. Kovse and T. Härder, 'Generic xmi-based uml model transformations', in *Object-Oriented Information Systems*, Z. Bellahsène, D. Patel and C. Rolland, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 192–198, ISBN: 978-3-540-46102-9.

[12] R. Wei, D. Kolovos, A. Garcia-Dominguez, K. Barmpis and R. Paige, 'Partial loading of xmi models', English, in *Proceedings - 19th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS 2016*, -, United States: ACM, Oct. 2016, pp. 329–339. DOI: 10.1145/2976767.2976787.

[13] W. Lu, K. Chiu and Y. Pan, 'A parallel approach to xml parsing', in *Proceedings of the 7th IEEE/ACM International Conference on Grid Computing*, ser. GRID '06, Washington, DC, USA: IEEE Computer Society, 2006, pp. 223–230, ISBN: 1-4244-0343-X. DOI: 10.1109/ICGRID.2006.311019. [Online]. Available: https://doi.org/10.1109/ICGRID.2006.311019.

[14] W. Lu and D. Gannon, 'Parallel xml processing by work stealing', in *Proceedings of the 2007 Workshop on Service-oriented Computing Performance: Aspects, Issues, and Approaches*, ser. SOCP '07, Monterey, California, USA: ACM, 2007, pp. 31–38, ISBN: 978-1-59593-717-9. DOI: 10.1145/1272457.1272462. [Online]. Available: http://doi.acm.org/10.1145/1272457.1272462.

[15] Y. Pan, Y. Zhang, K. Chiu and W. Lu, 'Parallel xml parsing using meta-dfas', in *Proceedings of the Third IEEE International Conference on e-Science and Grid Computing*, ser. E-SCIENCE '07, Washington, DC, USA: IEEE Computer Society, 2007, pp. 237–244, ISBN: 0-7695-3064-8. DOI: 10.1109/E-SCIENCE.2007.55. [Online]. Available: http://dx.doi.org/10.1109/E-SCIENCE.2007.55.

[16] B. Shah, P. R. Rao, B. Moon and M. Rajagopalan, 'A data parallel algorithm for xml dom parsing', in *Database and XML Technologies*, Z. Bellahsène, E. Hunt, M. Rys and R. Unland, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 75–90, ISBN: 978-3-642-03555-5.

[17] *Unified modelling language*, http://www.uml.org/index.htm, Accessed: 2019-03-07, 2019.

[18] *XML Syntax Rules*, https://www.ibm.com/support/knowledgecenter/en/SSRJDU/reference/SCN_XML_Syntax_Rules.html, Accessed: 2019-02-20.

[19] T. J. Grose, G. C. Doney and S. A. Brodsky, *Mastering XMI: Java Programming with XMI, XML and UML*. New York, NY, USA: John Wiley & Sons, Inc., 2001, ISBN: 0471384291.

[20] F. Wang, J. Li and H. Homayounfar, 'A space efficient xml dom parser', *Data & Knowledge Engineering*, vol. 60, no. 1, pp. 185–207, 2007, Intelligent Data Mining, ISSN: 0169-023X. DOI: https://doi.org/10.1016/j.datak.2006.01.002. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0169023X06000024.

[21] H. J. and G. M., 'Xml parsers - a comparative study with respect to adaptability', PhD thesis, University of Skövde, 2018.

[22] M. G. Kostoulas, M. Matsa, N. Mendelsohn, E. Perkins, A. Heifets and M. Mercaldi, 'Xml screamer: An integrated approach to high performance xml parsing, validation and deserialization', in *Proceedings of the 15th International Conference on World Wide Web*, ser. WWW '06, Edinburgh, Scotland: ACM, 2006, pp. 93–102, ISBN: 1-59593-323-9. DOI: 10.1145/1135777.1135796. [Online]. Available: http://doi.acm.org/10.1145/1135777.1135796.

[23] *XMIReader Package Overview*, https://www.sdmetrics.com/javadoc/com/sdmetrics/model/XMIReader.html, Accessed: 2019-02-12, 2019.

[24] *ArgoUML*, http://freshmeat.sourceforge.net/projects/argouml, Accessed: 2019-02-12, 2009.

[25] *Eclipse EMF*, https://github.com/eclipse/emf/tree/master/plugins/org.eclipse.emf.ecore.xmi/src/org/eclipse/emf/ecore/xmi/impl, Accessed: 2019-02-12, 2018.

[26] G. S. Almasi and A. Gottlieb, *Highly Parallel Computing*. Redwood City, CA, USA: Benjamin-Cummings Publishing Co., Inc., 1989, ISBN: 0-8053-0177-1.

[27] M. I. Gordon, W. Thies and S. Amarasinghe, 'Exploiting coarse-grained task, data, and pipeline parallelism in stream programs', *SIGPLAN Not.*, vol. 41, no. 11, pp. 151–162, Oct. 2006, ISSN: 0362-1340. DOI: 10.1145/1168918.1168877. [Online]. Available: http://doi.acm.org/10.1145/1168918.1168877.

[28] *ExecutorService - Oracle Docs*, https://docs.oracle.com/javase/7/docs/api/java/util/concurrent/ExecutorService.html, Accessed: 2019-03-20, 2008.

[29] D. Steinberg, F. Budinsky, M. Paternostro and E. Merks, *EMF: Eclipse Modeling Framework 2.0*, 2nd. Addison-Wesley Professional, 2009, ISBN: 0321331885.

[30] M. Nicola and J. John, 'Xml parsing: A threat to database performance', in *Proceedings of the Twelfth International Conference on Information and Knowledge Management*, ser. CIKM '03, New Orleans, LA, USA: ACM, 2003, pp. 175–178, ISBN: 1-58113-723-0. DOI: 10.1145/956863.956898. [Online]. Available: http://doi.acm.org/10.1145/956863.956898.

[31] S. Campanoni and G.-Y. Wei, 'Breaking cyclic-multithreading parallelization with xml parsing', 2014.

[32] D. P. Rodgers, 'Improvements in multiprocessor system design', in *Proceedings of the 12th Annual International Symposium on Computer Architecture*, ser. ISCA '85, Boston, Massachusetts, USA: IEEE Computer Society Press, 1985, pp. 225–231, ISBN: 0-8186-0634-7. [Online]. Available: http://dl.acm.org/citation.cfm?id=327010.327215.

[33] W3C, *Extensible Markup Language (XML) 1.0 (Fifth Edition)*, https://www.w3.org/TR/xml/, Accessed: 2019-03-02, 2008.

[34] J. A. Lukes, 'Efficient algorithm for the partitioning of trees', *IBM Journal of Research and Development*, vol. 18, no. 3, pp. 217–224, May 1974. DOI: 10.1147/rd.183.0217.

[35] OMG, *XML Metadata Interchange (XMI) Specification*, https://www.istr.unican.es/pyemofuc/PyEmofUCFiles/XMI_formal-14-04-04.pdf, Accessed: 2019-03-02, 2014.

[36] Sun Microsystems, Inc., *Multithreaded Programming Guide*. 4150 Network Circle Santa Clara, CA 95054 U.S.A., 2008, ch. 8, https://docs.oracle.com/cd/E19120-01/open.solaris/816-5137/compat-83941/index.html, Accessed: 2019-03-20.

[37] C. A. R. Hoare, 'Monitors: An operating system structuring concept', *Commun. ACM*, vol. 17, no. 10, pp. 549–557, Oct. 1974, ISSN: 0001-0782. DOI: 10.1145/355620.361161. [Online]. Available: http://doi.acm.org/10.1145/355620.361161.

[38] E. Merks, *Eclipse Community Forums: EMF*. [Online]. Available: https://www.eclipse.org/forums/index.php?t=msg&th=1098264&goto=1804720&#msg_1804720/ (Accessed: 04/04/2019).

[39] VisualVM, *VisualVM*, 2017. [Online]. Available: https://visualvm.github.io/ (Accessed: 15/04/2019).

[40] IBM, *IBM SDK, Java Technology Edition, Version 8*. 2019, ch. J9 VM Reference, https://www.ibm.com/support/knowledgecenter/en/SSYKE2_8.0.0/com.ibm.java.vm.80.doc/docs/jit_overview.html, Accessed: 2019-04-15.

[41] Justin Gesso, *Java Memory Management for Java Virtual Machine (JVM)*, 2017. [Online]. Available: https://betsol.com/2017/06/java-memory-management-for-java-virtual-machine-jvm/ (Accessed: 15/04/2019).